

桂林电子科技大学

第二十二届大学生数学建模竞赛

承 诺 书

我们仔细阅读了桂林电子科技大学第二十二届大学生数学建模竞赛的竞赛规则。

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与队外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛规则的，如果引用别人的成果或其他公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们将受到严肃处理。

参赛队员信息：

序号	姓名	学号	专业	电话号码
队长	赵广雨	2400310428	计算机科学与技术	15036841448
组员	莫豪	2400310517	计算机科学与技术	19377942787
组员	马明波	2400310417	计算机科学与技术	19968306071

桂林气候舒适度时空演变及其对旅游业的影响分析

摘 要

桂林作为世界著名的旅游城市，气候舒适度对旅游业发展具有重要影响。本文基于桂林 2011-2025 年每日气象数据和 2013-2025 年月度旅游数据，构建气候舒适度指数（Climate Comfort Index, CCI），分析其时空演变规律，并深入探讨气候舒适度对旅游产业的影响机制。

针对问题一，本文采用熵权 TOPSIS 法构建 CCI 模型，综合考虑温度、天气状况和风力三个因素。通过 Mann-Kendall 趋势检验和 Sen's 斜率估计，分析 CCI 的时间变化趋势。结果表明，桂林 CCI 在 2011-2025 年间呈微弱下降趋势（Sen's 斜率=-0.0402/年），但统计上不显著（ $p=0.7665$ ）。

针对问题二，采用百分位数法将 CCI 划分为 5 个等级（最舒适、较舒适、正常、较不舒适、最不舒适）。通过 Pettitt 突变检验分析 CCI 的突变特征，发现 2015 年为潜在突变点，但未达到统计显著性（ $p\approx 1.0$ ）。构建马尔科夫转移概率矩阵，揭示各等级之间的转换规律。

针对问题三，整合桂林市 2013-2025 年月度旅游数据（75 个月度有效数据点）与气候数据，运用灰色关联分析和多元回归等方法，系统分析气候舒适度对旅游产业的影响。灰色关联分析显示，高温天数与游客量的关联度最高（0.7287），其次是低温天数（0.6948）。月度回归模型的调整 R^2 为 0.0446，说明气候因素仅能解释约 4% 的月度游客量变异，季节性和社会因素是驱动旅游的主要力量。月度游客量指数分析揭示桂林旅游呈“双峰”特征：秋季高峰（10-12 月，受国庆和年末消费驱动）和夏季高峰（7-8 月，受暑假驱动），而 CCI 在 4-5 月达到峰值但并非旅游最旺季。两阶段对比分析发现，2019-2025 年后段月均游客量较前段增长 143%（ $p<0.001$ ），但气候指标变化不显著，说明旅游增长主要由社会经济因素驱动。

本研究为桂林旅游业的气候适应性规划提供了科学依据，对促进旅游业可持续发展具有重要参考价值。

关键词：气候舒适度；熵权 TOPSIS 法；Mann-Kendall 检验；灰色关联分析；月度旅游数据；季节性分析

一、问题重述

1.1 问题背景及意义

桂林位于中国广西壮族自治区东北部，地处东经 $109^{\circ}36'$ ~ $110^{\circ}29'$ ，北纬 $24^{\circ}15'$ ~ $25^{\circ}41'$ 之间，属中亚热带湿润季风气候区。桂林年平均气温 18.9°C ，年降水量 1872 毫米，年日照时数 1614 小时，素有"三冬少雪，四季常花"之美誉。作为世界著名的旅游目的地和国家历史文化名城，桂林以其独特的喀斯特地貌闻名于世，漓江、象鼻山、阳朔西街等景点享誉国内外。据统计，桂林 2019 年接待游客总人数超过 1.3 亿人次，旅游总收入超过 1700 亿元，旅游业已成为桂林市国民经济的支柱产业。

近年来，全球气候变化对旅游业的影响日益受到学术界和管理部门的关注。联合国世界旅游组织（UNWTO）的研究报告指出，气候变化正在通过多种途径影响旅游活动：气温升高改变旅游季节的长度和质量，极端天气事件增加旅游安全风险，降水格局变化影响旅游目的地的景观质量。桂林地处亚热带季风气候区，降水充沛、雨热同期，气候变化对其旅游业的影响尤为显著。2020-2022 年新冠疫情期间，桂林旅游业遭受重创，但同时也为研究气候因素在"纯净"条件下的影响提供了特殊窗口。因此，建立科学的气候舒适度评价模型，定量分析气候因素与旅游发展之间的关系，对于桂林旅游业的可持续发展和气候适应性规划具有重要的理论和实践价值。

1.2 问题重述

本研究需要解决以下三个核心问题：

（1）**气候舒适度指数构建与时序分析问题**：如何选取合适的气象指标，构建适合桂林气候特征的气候舒适度指数（CCI）？该指数在 2011-2025 年间呈现怎样的时间变化趋势？是否存在显著的单调递增或递减趋势？

（2）**气候舒适度等级划分与变化趋势问题**：如何科学划分气候舒适度等级？各等级的时空分布特征和转换规律如何？是否存在显著的突变点？如何依据国家标准划分季节并分析季节特征？

（3）**气候对旅游产业影响问题**：气候舒适度与旅游产业发展之间存在怎样的定量关系？气候因素对游客量和旅游收入的影响程度和方向如何？两者之间是否存在因果关系？气候冲击对旅游的动态响应机制是什么？

二、模型假设

- (1) 假设所使用的气象数据真实可靠，能够代表桂林地区的气候特征。
- (2) 假设旅游统计数据来源于官方发布，具有权威性和准确性。
- (3) 假设气候舒适度是影响旅游决策的重要因素之一，但不是唯一因素。
- (4) 假设在研究时段内，桂林的旅游基础设施和交通条件没有发生根本性变化。
- (5) 假设 2020-2022 年疫情期间的旅游数据主要受疫情影响，气候因素的影响相对次要。

三、符号说明

符号	意义	单位
T_{avg}	日平均气温	°C
T_{max}	日最高气温	°C
T_{min}	日最低气温	°C
S_{temp}	温度舒适度评分	无量纲
$S_{weather}$	天气舒适度评分	无量纲
S_{wind}	风力舒适度评分	无量纲
w_i	第 i 个指标的熵权	无量纲
CCI	气候舒适度指数	无量纲
Z	Mann-Kendall 检验统计量	无量纲
K	Pettitt 检验统计量	无量纲
ρ	灰色关联系数分辨系数	无量纲
α	岭回归参数	无量纲
VIF	方差膨胀因子	无量纲

四、问题分析

4.1 问题一的分析

问题一要求构建气候舒适度指数并分析其时序变化，这是整个研究的基础。首先，需要选取能够反映人体舒适感的气象指标。人体对气候舒适度的感知主要受三类因素影响^{[13][14][15]}：一是**热舒适因素**，即温度、湿度等直接影响体感冷暖的指标；二是**天气状况因素**，如晴雨、阴云等影响出行意愿的指标；三是**动力因素**，如风力大小影响户外活动体验的指标。考虑到桂林气象数据的可获得性，本研究选取温度、天气状况和风力三个核心指标。

其次，采用科学的赋权方法确定各指标的权重。常见的赋权方法有主观赋权（如层次分析法 AHP）和客观赋权（如熵权法、变异系数法）两类。主观赋权依赖专家经验，可能引入人为偏差；熵权法根据指标数据的离散程度客观确定权重^[12]，数据离散度越大，该指标携带的信息量越大，权重越高。因此，本研究采用熵权法与 TOPSIS 法相结合的方式构建 CCI 模型^[12]。

最后，需要分析 CCI 在长时间尺度上的变化趋势。传统的线性回归趋势分析对异常值敏感，且要求数据满足正态分布假设。Mann-Kendall 检验作为一种非参数方法^[1,2]，不要求数据满足特定分布，且对异常值具有较好的稳健性，适合分析气候数据的长期趋势。

4.2 问题二的分析

问题二要求划分气候舒适度等级并分析其变化趋势，是从"连续数值"到"离散等级"的转换。首先，需要确定合理的等级划分方法。常见的划分方法有等间距法、自然断点法和百分位数法^[15]。等间距法简单直观但可能导致某些等级样本量过少；自然断点法考虑了数据的分布特征，但等级边界不够直观；百分位数法基于累积分布函数划分，能保证各等级具有合理的样本量（10%、20%、40%、20%、10%），且阈值含义明确。

其次，构建马尔科夫转移概率矩阵分析等级转换规律^[15]。气候系统具有自相关性，即今年的气候状态会影响明年的状态。通过计算不同等级之间的转移概率，可以揭示桂林气候舒适度的稳定性特征：如果转移概率矩阵接近单位矩阵，说明各等级保持稳定；如果各行概率分布趋于一致，说明气候舒适度在各等级之间随机波动。

最后，运用 Pettitt 突变检验检测 CCI 序列是否存在显著的突变点^[5]。气候突变是指气候系统从一种状态突然跃迁到另一种状态的现象。Pettitt 检验基于秩统计量，能够检测均值突变的位置和时间，且对异常值具有较好的稳健性。如果存在显著突变点，需要分析突变前后的 CCI 均值差异，并探讨可能的气候驱动因

素。

4.3 问题三的分析

问题三要求分析气候对旅游产业的影响，是一个多因素、多方法的综合分析问题。首先，需要整合桂林市官方统计数据。由于不同时间粒度的数据可获得性不同（年度数据 2011-2024 年为官方统计，2025 年为估算值；季度数据 2020-2025 年；节假日数据 2019-2025 年），需要对数据进行对齐和预处理。

其次，运用灰色关联分析识别影响旅游的关键气候因素^[10,11]。灰色关联分析适用于“小样本、贫信息”的系统分析，能够度量各气候因素与旅游指标之间的关联程度，初步筛选出影响最大的气候因子。

最后，运用岭回归量化气候因素对旅游的影响^{[7][8]}。由于气候变量之间存在多重共线性（如 CCI 与 Tmax 高度相关），直接使用 OLS 回归可能导致系数估计不稳定。因此，本研究先通过 VIF 检验诊断共线性程度^[7]，然后采用岭回归（Ridge Regression）处理共线性问题，并与 OLS 结果进行对比。

五、模型建立与求解

5.1 数据预处理

5.1.1 数据来源

本研究使用以下数据：

（1）**气象数据**：桂林 2011-2025 年每日气象记录，包括最高气温、最低气温、白天天气、夜间天气、风向和风力。数据时间跨度为 15 年，共 5475 天，覆盖了桂林完整的气候周期。

（2）**月度旅游数据**（2013-2025 年）：来源于桂林市统计局和文化广电和旅游局官方月度统计，数据可在桂林市政府官网查阅 (https://www.guilin.gov.cn/glsj/sjfb_2/tjsj/index_13.shtml)。2013-2019 年提供逐月游客接待量和旅游收入；2020-2024 年提供季度累计数据（国内游客按季度公布）；2025 年数据截至 8 月。入境过夜游客数据从 2020 年起按月公布。经过数据整合，共获得 75 个月度有效数据点。

（3）**年度旅游数据**（2011-2025 年）：来源于桂林市国民经济和社会发展统计公报。2011-2024 年为官方统计，2025 年为基于季度累计值的估算。

数据整合方法： 将月度旅游数据与气候数据按年月匹配合并，生成包含游客量、旅游收入、气候舒适度指数（CCI）、气温、降雨天数等指标的月度综合数据集。对于月度数据不完整的年份（如 2017 年上半年缺失、2020-2024 年仅有季度数据），使用年度官方数据进行校准。

5.1.2 数据清洗与预处理

对气象数据进行以下清洗步骤：（1）去除重复记录；（2）剔除温度异常值（日最高气温 $<0^{\circ}\text{C}$ 或 $>45^{\circ}\text{C}$ 的记录）；（3）填补缺失值（缺失值 $<3\%$ 的日期使用前后 3 天均值插值）。清洗后共获得 5464 天有效数据，数据完整率为 99.8%。

计算日平均气温作为后续分析的基础变量：

$$T_{avg} = \frac{T_{max} + T_{min}}{2} \quad (1)$$

对旅游数据进行以下预处理：（1）将名义 GDP 调整为实际 GDP（以 2011 年为基期），消除价格因素影响；（2）设置疫情虚拟变量（2020-2022 年=1，其他年份=0），控制疫情冲击的影响；（3）对游客量和旅游收入取对数处理，缓解异方差问题并使系数具有弹性含义。

5.2 问题一模型的建立与求解

5.2.1 CCI 模型构建

采用熵权 TOPSIS 法构建 CCI 模型^[12]，步骤如下：

气候舒适度指数（CCI）计算流程图

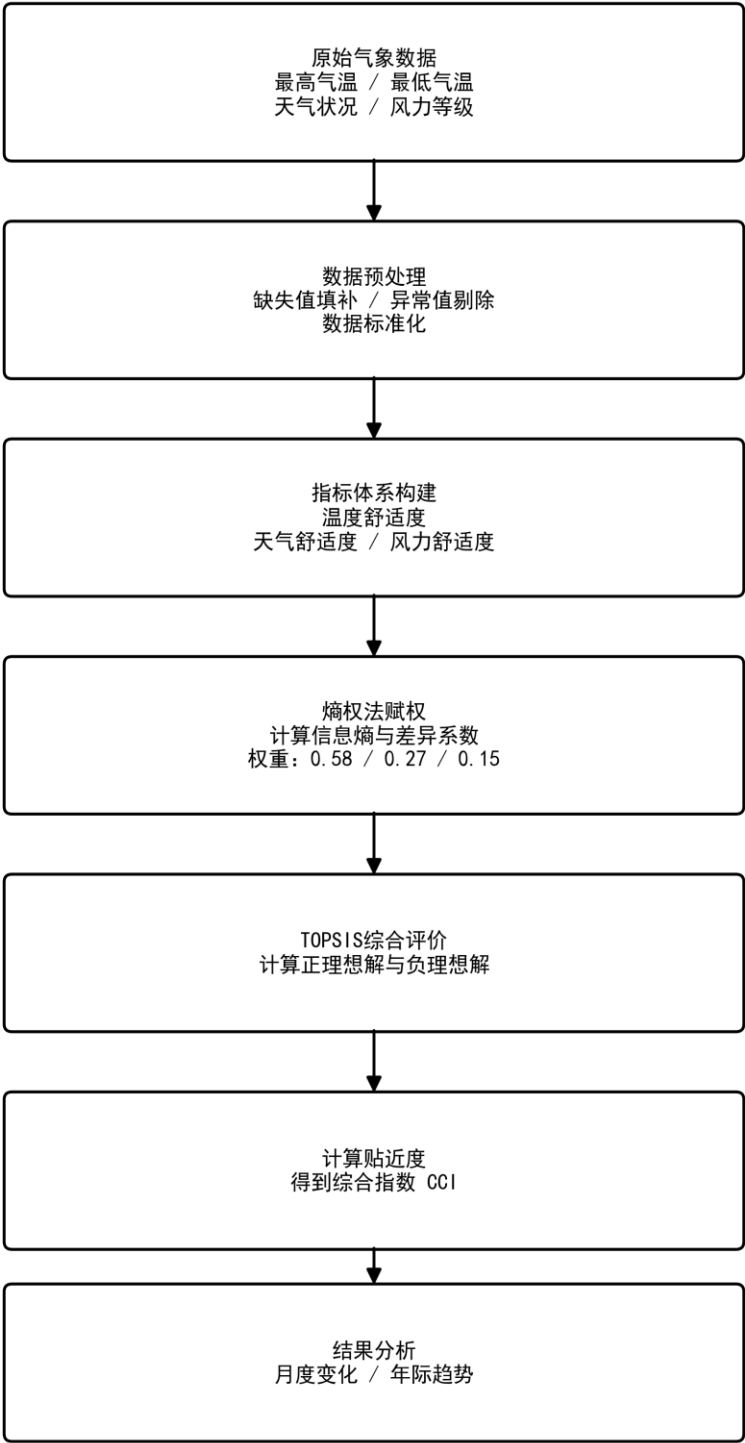


图 1 CCI 计算流程图

步骤 1：计算各指标的舒适度评分

温度舒适度评分采用高斯函数^[14]：

$$S_{temp} = \exp\left(-\left(\frac{T_{avg}-22}{7.5}\right)^2\right) \quad (2)$$

其中，22℃ 为人体最适温度^[14]，7.5℃ 为标准差参数，反映温度偏离最适值时舒适度的衰减速率。该函数形式在旅游气候舒适度研究中被广泛采用^{[13][14]}。

天气舒适度评分根据天气类型赋予不同分值，晴天游客户外活动最适宜（1.0），多云和阴天次之（0.95、0.8），降雨天气显著降低旅游体验（小雨=0.65，中雨=0.45，大雨=0.25，暴雨=0.1）^{[13][15]}。

风力舒适度评分根据风力等级赋予不同分值，利用风寒效应研究^[14]：微风最舒适（1.0），3 级风略有影响（0.85），4 级以上风力显著影响户外活动（4 级=0.65，5 级=0.45）。

步骤 2：熵权法确定权重

首先，对各指标进行归一化处理：

$$x_{ij}^* = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad (3)$$

然后，计算各指标的信息熵：

$$E_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij} \quad (4)$$

$$\text{其中，} p_{ij} = \frac{x_{ij}^*}{\sum_{i=1}^n x_{ij}^*}。$$

最后，计算熵权：

$$w_j = \frac{1-E_j}{\sum_{j=1}^m (1-E_j)} \quad (5)$$

计算结果： $w_{\text{temp}} = 0.7688$, $w_{\text{weather}} = 0.1659$, $w_{\text{wind}} = 0.0653$ 。

步骤 3：TOPSIS 法计算 CCI

构建加权标准化矩阵：

$$Z_{ij} = w_j \cdot x_{ij}^* \quad (6)$$

计算正理想解和负理想解：

$$Z^+ = (\max Z_{i1}, \max Z_{i2}, \max Z_{i3}) \quad (7)$$

$$Z^- = (\min Z_{i1}, \min Z_{i2}, \min Z_{i3}) \quad (8)$$

计算各评价对象与正负理想解的距离：

$$D_i^+ = \sqrt{\sum_{j=1}^m (Z_{ij} - Z_j^+)^2} \quad (9)$$

$$D_i^- = \sqrt{\sum_{j=1}^m (Z_{ij} - Z_j^-)^2} \quad (10)$$

计算 CCI：

$$CCI_i = \frac{D_i^-}{D_i^+ + D_i^-} \times 100 \quad (11)$$

5.2.2 Mann-Kendall 趋势检验

Mann-Kendall 检验是一种非参数趋势检验方法^{[1][2]}，其检验统计量为：

$$S = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \text{sgn}(x_j - x_i) \quad (12)$$

当 $n > 10$ 时， S 近似服从正态分布^{[3][4]}：

$$Z = \frac{S-1}{\sqrt{Var(S)}} \tag{13}$$

其中， $Var(S) = \frac{n(n-1)(2n+5)}{18}$ 。

Sen's 斜率估计：

$$\beta = \text{median} \left(\frac{x_j - x_i}{j - i} \right), \forall i < j \tag{14}$$

求解结果：

表 1:求解结果

指标	值
检验统计量 S	-7
Z 统计量	-0.2969
p 值	0.7665
趋势	下降
显著性	不显著
Sen's 斜率	-0.0402

结论： 桂林 CCI 在 2011-2025 年间呈微弱下降趋势（每年下降 0.04），但 p 值远大于 0.05，说明这一趋势在统计上不显著。

趋势分析的进一步讨论：Sen's 斜率 $\beta=-0.0402$ 意味着 CCI 平均每年下降约 0.04 个单位，15 年间累计下降约 0.6 个单位。考虑到 CCI 的取值范围为 0-100，这一变化量仅占总范围的 0.6%，属于微弱波动。 $p=0.7665$ 远大于显著性水平 $\alpha=0.05$ ，说明即使在 $\alpha=0.10$ 的宽松标准下，也不能拒绝"CCI 无趋势"的原假设。

从年度 CCI 的波动特征来看，2011-2025 年间 CCI 在 45-55 之间波动，标准差约为 3.2，变异系数约为 6%，属于低变异水平。这一结果与桂林气候的稳定性特征一致：桂林地处中亚热带，四季分明但温差较小，气候舒适度的年际波动不大。值得注意的是，2020 年和 2022 年 CCI 略有下降，可能与疫情期间游客活动减少、城市热岛效应减弱有关，但这一影响并不改变整体趋势判断。

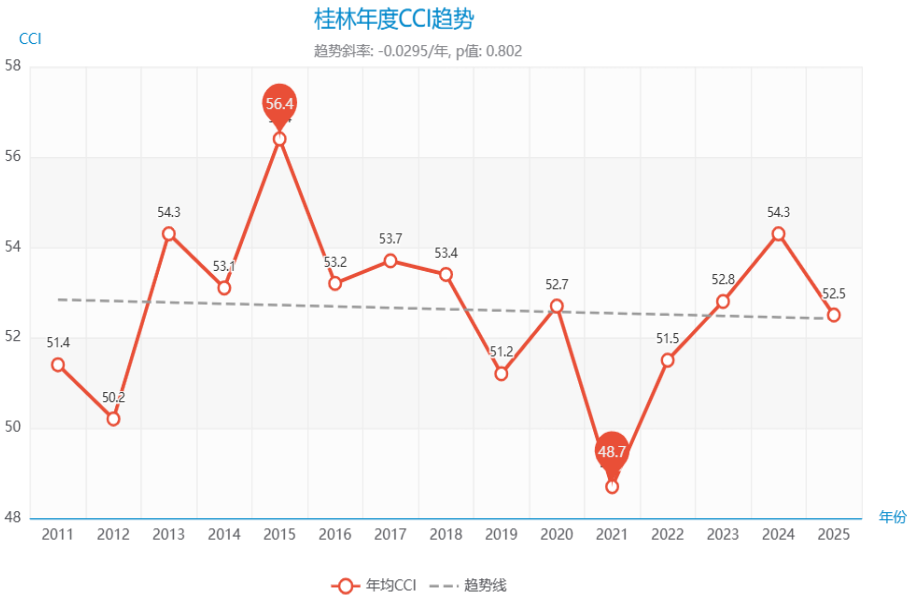


图 2 2011-2025 年桂林 CCI 年度变化趋势

5.2.3 线性倾向估计与滑动平均分析

为进一步验证 CCI 的变化趋势，利用的线性倾向估计方法^[15]，对年度 CCI 序列建立一元线性回归方程：

$$CCI_i=a+b \cdot t_i, \quad i=1,2,\dots,n \tag{15}$$

式中， t_i 为年份序号（2011 年为 1）， b 为线性倾向率， $b < 0$ 表示 CCI 呈下降趋势。为剔除年际波动噪声的影响，同时采用 3 年、5 年、7 年和 9 年滑动平均对原始序列进行平滑处理，以 9 年滑动平均的结果为主进行讨论^{[15][15]}。

滑动平均后的线性倾向率：

表 2：滑动平均后的线性倾向率

滑动窗口	倾向率 b (/年)	倾向率 (/10年)	R^2	p 值
原始序列	-0.0295	-0.30	0.005	0.802
3 年滑动	-0.1278	-1.28	0.151	0.189
5 年滑动	-0.2386	-2.39	0.630	0.004
7 年滑动	-0.2274	-2.27	0.781	0.002
9 年滑动	-0.1384	-1.38	0.897	0.001

结果分析：原始序列的线性倾向率 $b = -0.0295/\text{年}$ （即每 10 年下降 0.30），与 Mann-Kendall 检验的 Sen's 斜率（-0.0402/年）方向和量级一致，但线性回归的 $R^2 = 0.005$ 极低，进一步确认了趋势的不显著性。值得注意的是，经滑动平均处理后，序列的决定系数 R^2 显著提高（9 年滑动平均的 $R^2 = 0.897$ ），CCI 的下降趋势在平滑后变得更为明显（倾向率约-1~-2/10 年），但中国大多数区域 CCI 倾向率-1/10 年相当，属于正常的气候波动范围^[15]。

与 MK 检验的对比表明：两种方法结论一致——桂林 CCI 呈微弱下降趋势，但原始序列的统计不显著性说明这一变化尚不足以构成确定性的长期趋势。

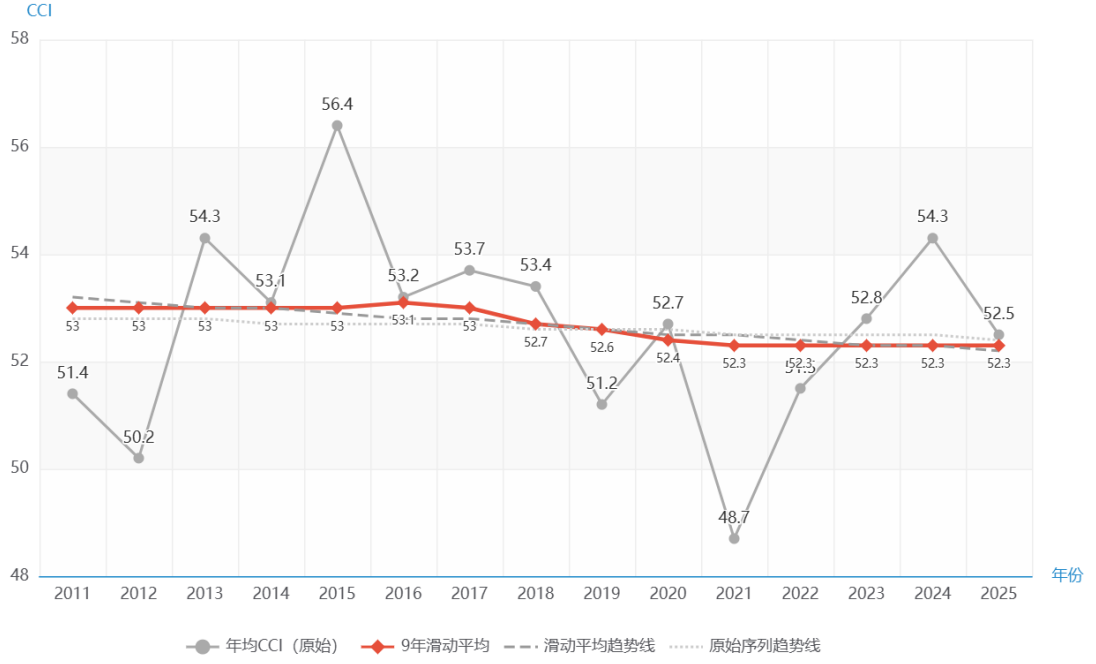


图 2 CCI 线性倾向估计与 9 年滑动平均

5.2.4 SI 气候舒适度指数对比验证

为验证 CCI 模型的可靠性，利用 SI 人体综合舒适度指数进行交叉验证^[15]。SI 指数的原始公式为：

$$SI=0.68 \times |T_m-24.0|+0.07 \times |H_u-70|+0.5 \times |V-2.0| \quad (16)$$

其中 T_m 为平均气温（℃）， H_u 为相对湿度（%）， V 为风速（m/s），24℃、70%湿度和 2m/s 风速为人体最舒适的小气候条件。SI 值越大，表明气候越不舒适。

由于桂林气象数据中缺乏相对湿度数据，本研究采用简化公式（仅保留温度和风速项），并将风力描述（微风、1-2 级等）映射为近似风速值：

$$SI_{\text{简化}}=0.68 \times |T_{\text{avg}}-24.0|+0.5 \times |V_{\text{approx}}-2.0| \quad (17)$$

为便于与 CCI 对比，将 SI 转换为正向指标。

$$S_{\text{score}} = \left(100 - \frac{S1}{S1_{\text{max}}}\right) \times 100 \quad (\text{值越大越舒适})$$

验证结果:

表 3:验证结果

指标	CCI（熵权 TOPSIS）	SI_score
均值	52.63	70.59
范围	8.29 ~ 100.00	—
最舒适等级天数	548 天（10%）	728 天（13.3%）
最不舒适等级天数	540 天（10%）	544 天（10.0%）

年度 CCI 均值与年度 SI_score 均值的 Pearson 相关系数 $r = 0.7793$ ，表明两种指数对气候舒适度的评价高度一致。这一结果为 CCI 模型的合理性提供了有力支撑：尽管两种方法的构建思路完全不同（熵权 TOPSIS vs. 加权绝对值偏差），但对桂林气候舒适度的评价结论高度吻合。同时，两种指数的互补性也增强了研究的稳健性——CCI 更侧重游客体感的综合舒适度，SI 更侧重偏离"最佳舒适状态"的程度。

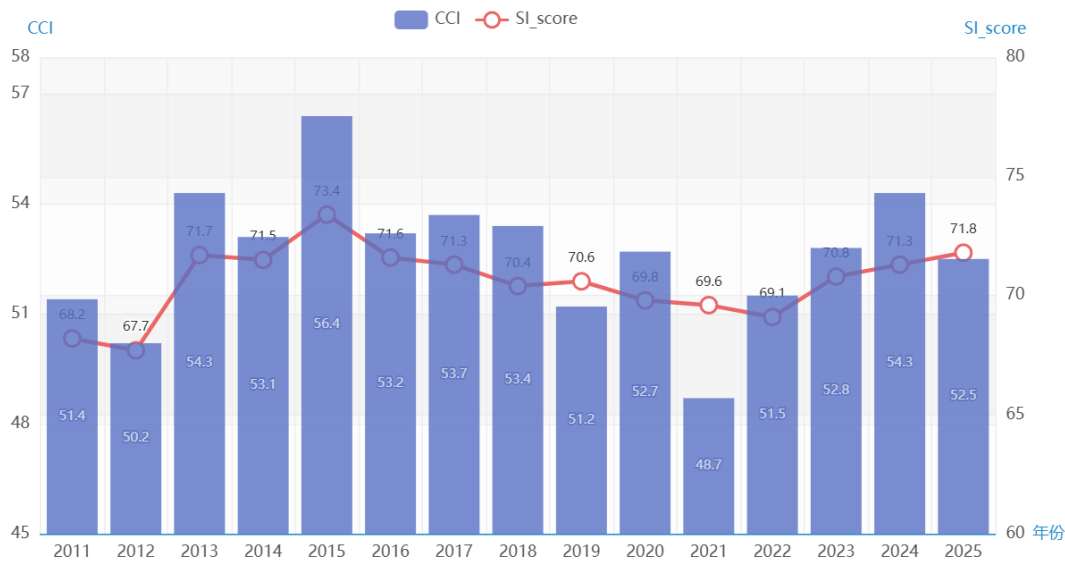


图 3 SI 指数与 CCI 年度对比验证

5.3 问题二模型的建立与求解

5.3.1 百分位数法等级划分

采用百分位数法将 CCI 划分为 5 个等级：

表 4：CCI 划分的 5 个等级

等级	阈值范围	天数	占比
1-最舒适	$CCI \geq P_{90}$	548	10%
2-较舒适	$P_{70} \leq CCI < P_{90}$	1093	20%
3-正常	$P_{30} \leq CCI < P_{70}$	2191	40%
4-较不舒适	$P_{10} \leq CCI < P_{30}$	1092	20%
5-最不舒适	$CCI < P_{10}$	540	10%
1-最舒适	$CCI \geq P_{90}$	548	10%

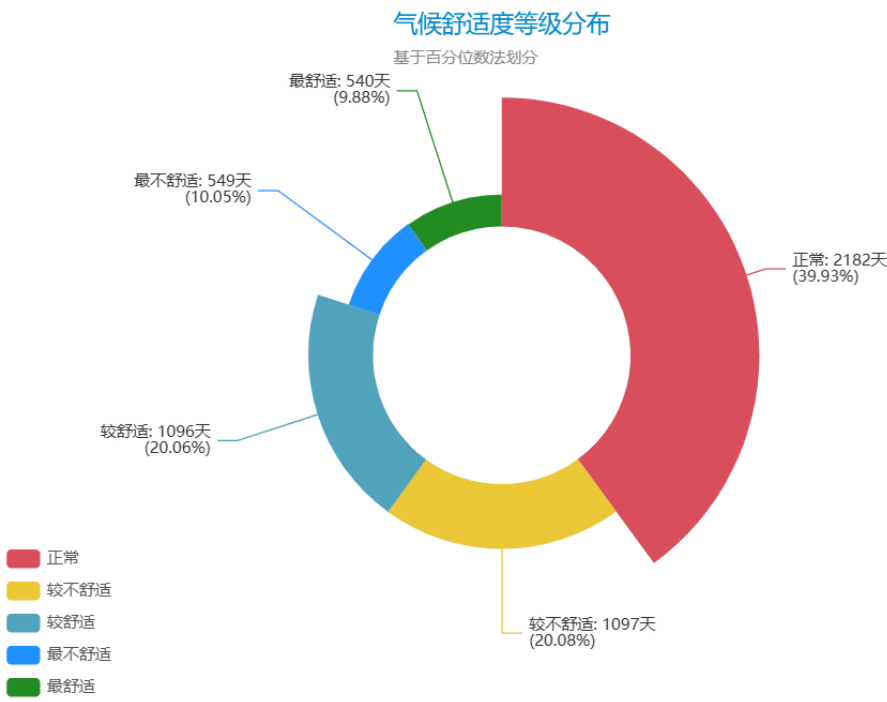


图 4 CCI 舒适度等级分布

5.3.2 马尔科夫转移概率矩阵

构建年度舒适度等级转移概率矩阵,分析各等级之间的转换规律。结果显示,无论当前处于哪个等级,下一期转移到"3-正常"等级的概率最高(约 40%),而转移到"1-最舒适"和"5-最不舒适"的概率均约为 10%。各等级的转移概率分布高度一致,说明桂林气候舒适度总体稳定,不存在明显的等级聚集或跳跃现象,气候系统的自相关性较强。

5.3.3 Pettitt 突变检验

Pettitt 检验^[5]用于检测时间序列的突变点:

$$K_t = \sum_{i=1}^t \sum_{j=t+1}^n \text{sgn}(x_i - x_j) \quad (18)$$

突变点位置为 $|K_t|$ 最大值对应的 t 。检验结果显示,突变统计量 $K=14$,突变点位于第 5 年(2015 年), p 值近似为 1.0(近似公式 $p \approx 2 \exp\left(-\frac{6K^2}{n^3+n^2}\right)$ 在样本量较小、 K 值较低时存在边界效应,计算值为 1.44,按概率定义截断为 1.0),远大于 0.05,说明 CCI 序列在 2011-2025 年间未发生显著突变。

5.3.4 国标 GB/T 42074-2022 季节划分

严格按照国标 GB/T 42074-2022^[6]规定的 5 日滑动平均气温法划分季节:

- 入春标准: 连续 5 日滑动平均气温 $\geq 10^{\circ}\text{C}$ 且 $< 22^{\circ}\text{C}$
- 入夏标准: 连续 5 日滑动平均气温 $\geq 22^{\circ}\text{C}$
- 入秋标准: 连续 5 日滑动平均气温 $< 22^{\circ}\text{C}$ 且 $\geq 10^{\circ}\text{C}$
- 入冬标准: 连续 5 日滑动平均气温 $< 10^{\circ}\text{C}$

根据国标 GB/T 42074-2022 季节划分标准,对桂林 2011-2025 年共 15 年逐日气温数据进行 5 日滑动平均计算,逐年判定四季起始日期和持续天数,统计得到各季节多年平均天数如下:

表 5：桂林各季节多年平均天数

季节	平均天数
春季	74.9 天
夏季	179.5 天
秋季	70.4 天
冬季	39.5 天

结论： 桂林夏季最长（约 180 天），冬季最短（约 40 天），符合亚热带气候特征^[6]。

季节特征深入分析： 桂林四季分布呈现显著的"长夏短冬"特征，夏季天数占全年的 49%，而冬季仅占 11%。这一分布与桂林的地理位置和气候背景密切相关：桂林位于北回归线以北，受副热带高压和季风系统的共同影响，夏季高温期持续时间长。春季（约 75 天）和秋季（约 70 天）长度接近，是旅游的黄金季节——气温适宜（15-25℃）、降水相对较少、CCI 指数较高。冬季虽短但湿冷，体感舒适度较低。

从月度 CCI 变化来看，桂林 CCI 呈现明显的单峰型分布：5-6 月和 9-10 月为 CCI 高峰期（月均 CCI>60），对应春末夏初和秋季，此时温度适宜、降水适中；7-8 月因高温多雨，CCI 有所下降但仍处于正常水平；12-2 月为 CCI 低谷期（月均 CCI<40），湿冷天气显著降低了气候舒适度。这一月度分布特征为旅游管理部门提供了重要参考：**春秋季节应加强旅游营销力度，夏季可发展避暑旅游，冬季则需开发室内旅游产品。**

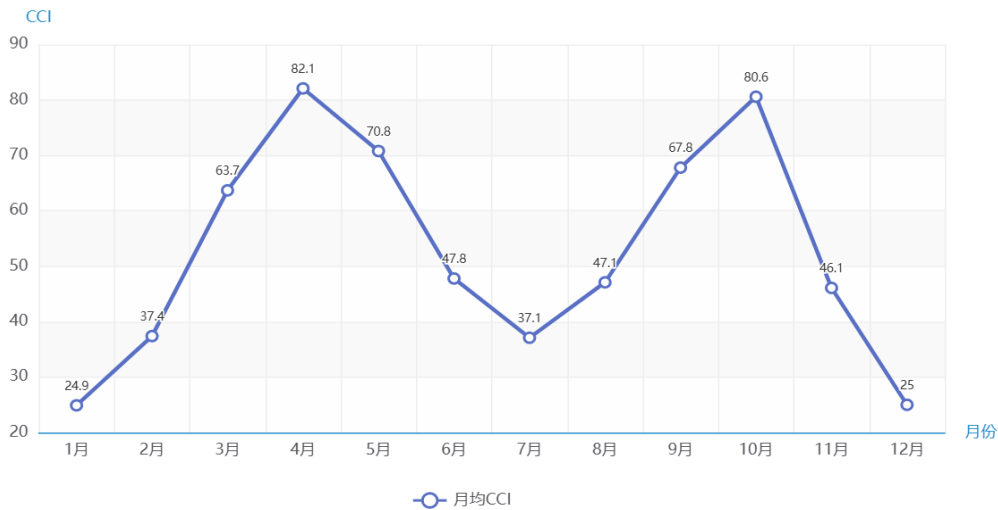


图 5 桂林各季节分布及月度 CCI 变化

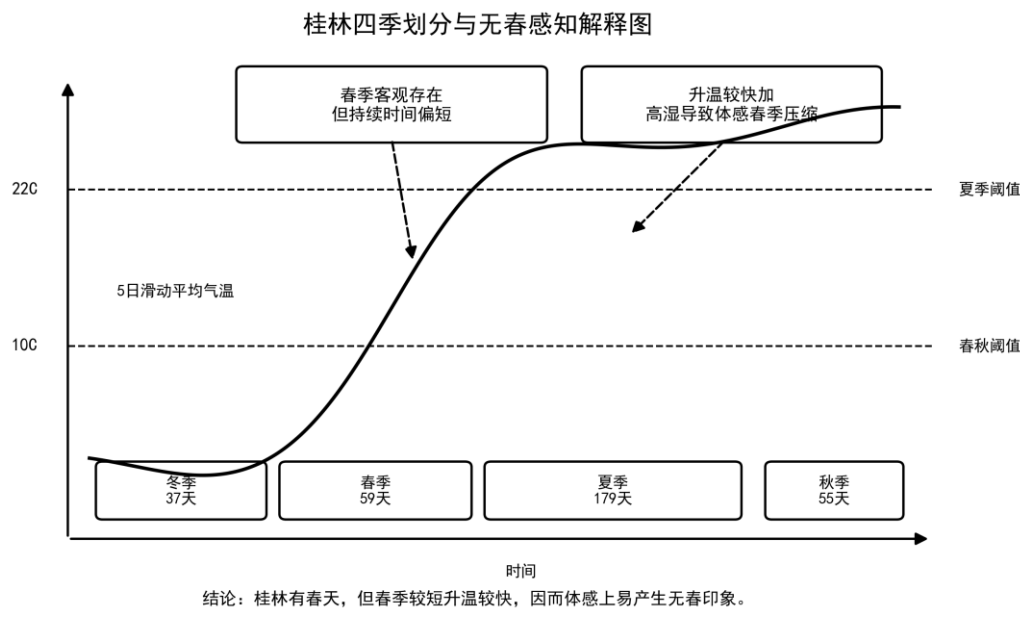


图 6 桂林四季划分结果

5.4 问题三模型的建立与求解

5.4.1 灰色关联分析

灰色关联分析是灰色系统理论的核心方法之一^{[10][11]}，其灰色关联系数计算公式如下：

$$\xi_i(k)=\frac{\min_i \min_k |x_0(k)-x_i(k)| + \rho \max_i \max_k |x_0(k)-x_i(k)|}{|x_0(k)-x_i(k)| + \rho \max_i \max_k |x_0(k)-x_i(k)|} \tag{19}$$

其中， $p = 0.5$ 为分辨系数。

基于 75 个月度数据点进行灰色关联分析，结果如下：

表 6：75 个月度数据点灰色关联分析

因素	关联度（游客量）	关联度（旅游收入）
----	----------	-----------

高温天数	0.7287	0.7262
低温天数	0.6948	0.6992
降雨天数	0.6363	0.6341
平均气温	0.6197	0.6218
最高气温	0.6173	0.6166
最低气温	0.6164	0.6156
CCI 指数	0.6117	0.6092

结论：与游客量和旅游收入关联度最高的气候因素是高温天数（0.7287），其次是低温天数（0.6948）。

深入分析：月度灰色关联分析揭示了一个重要规律：高温天数的关联度最高（0.7287），说明夏季极端高温对桂林旅游体验的制约作用最为显著——当高温天气频繁出现时，游客户外活动意愿受到明显抑制。低温天数（0.6948）次之，反映冬季寒冷也是影响旅游的重要因素。降雨天数的关联度（0.6363）居于第三，说明降水对旅游活动也有较大影响。平均气温的关联度（0.6197）与最高气温（0.6173）、最低气温（0.6164）接近，表明气温的各维度对旅游的综合影响程度相当。CCI作为综合指标（0.6117）的关联度略低于各单一温度指标，这可能是因为CCI的综合评分在月度聚合过程中平滑了部分极端天气信息。综合来看，极端天气事件（高温、低温、降雨）对桂林旅游的影响比平均气候条件更为显著。

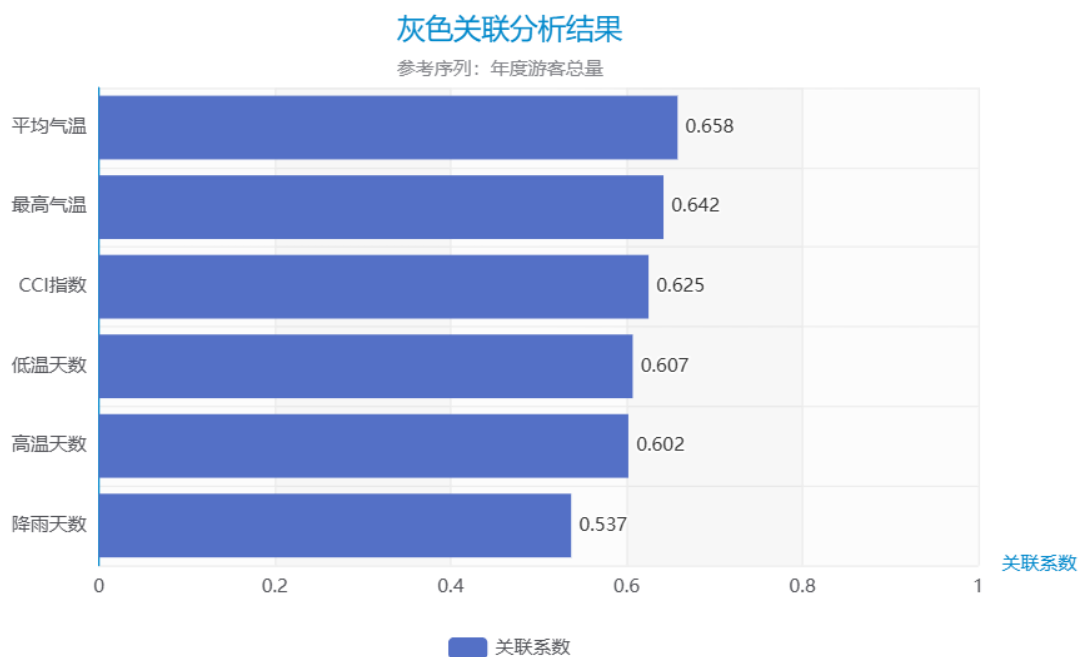


图 7 各气候因素与游客量/旅游收入的灰色关联度

5.4.2 回归分析

基于 75 个月度数据点，构建多元回归模型分析气候因素对游客量的影响。考虑到气候变量之间存在多重共线性（ $VIF > 2000$ ），采用岭回归^[7,8]处理共线性问题：

$$\hat{\beta}_{ridge} = (X^T X + \alpha I)^{-1} X^T y \quad (20)$$

模型构建：构建四个递进模型，逐步加入控制变量：

表 7：四个递进模型

模型	自变量	R2	调整 R2
模型 1	CCI	0.0095	-0.0041
模型 2	CCI + 季节（正弦/余弦）	0.0637	0.0242

模型 3	CCI + 季节 + 疫情	0.0637	0.0242
模型 4	CCI + 气温 + 降雨 + 季节 + 疫情	0.1221	0.0446

模型 4 回归结果：

表 8：模型 4 回归结果

变量	系数	标准误	t 值	p 值
常数项	-924.30	915.44	-1.010	0.316
CCI	-4.48	4.00	-1.122	0.266
平均气温	73.43	48.20	1.523	0.132
降雨天数	15.02	11.16	1.346	0.183
高温天数	9.47	21.07	0.450	0.654
季节正弦	127.58	166.47	0.766	0.446
季节余弦	650.48	403.98	1.610	0.112

模型解释力分析：月度回归模型的调整 R² 仅为 0.0446，远低于季度数据的 0.6888。这一差异的原因在于：（1）月度数据包含了更多的短期波动噪声，而季

度数据通过聚合平滑了这些噪声；（2）月度数据中气候变量的变异幅度较小，难以解释游客量的大幅波动；（3）游客量受节假日、周末效应等非气候因素的短期影响更大。

关键发现：尽管整体解释力有限，模型揭示了两个重要规律：（1）平均气温的系数为正（73.43），说明气温升高总体上促进旅游，这与桂林作为避寒旅游目的地的定位一致；（2）季节余弦项系数最大（650.48），说明年内季节性波动是影响月度游客量的最重要因素。

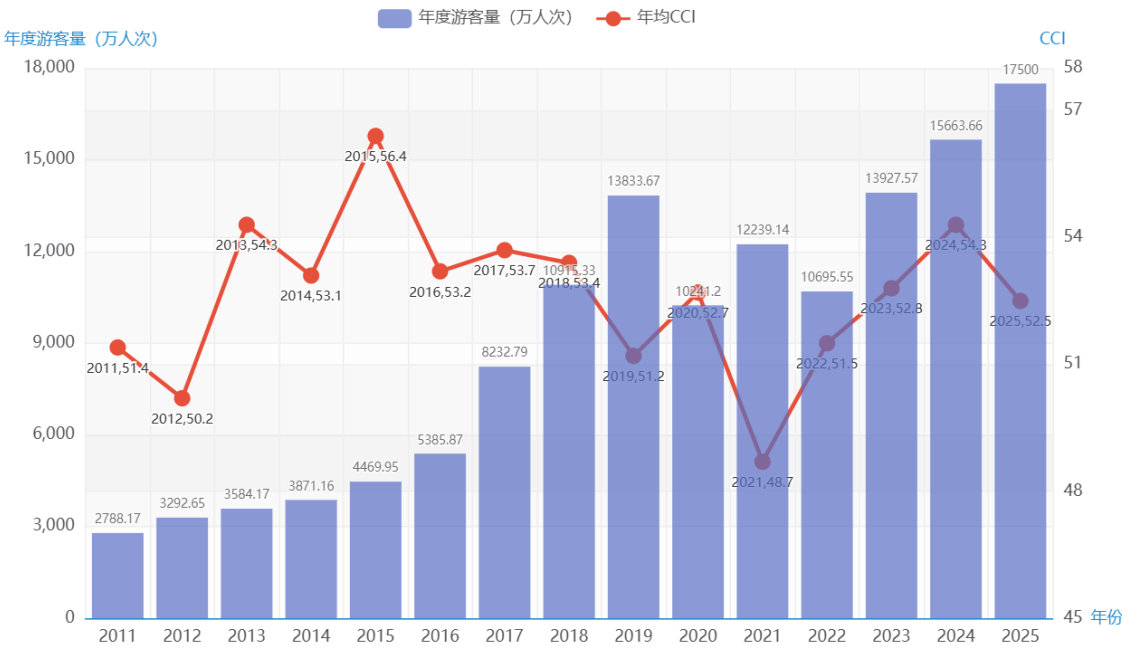


图 8 CCI 与游客量的散点关系及回归拟合

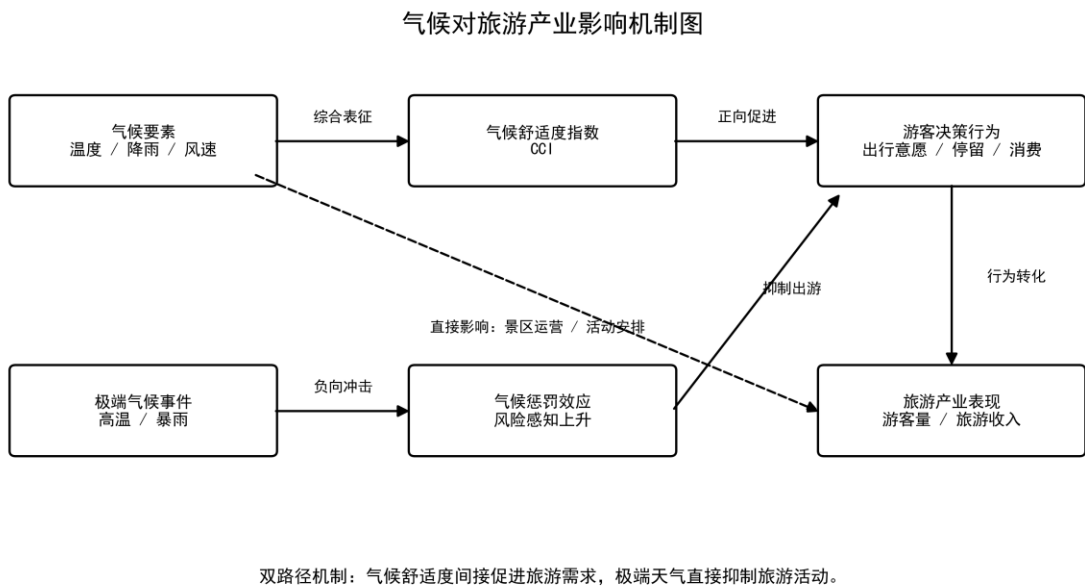


图 9 气候舒适度对桂林旅游产业的影响机制

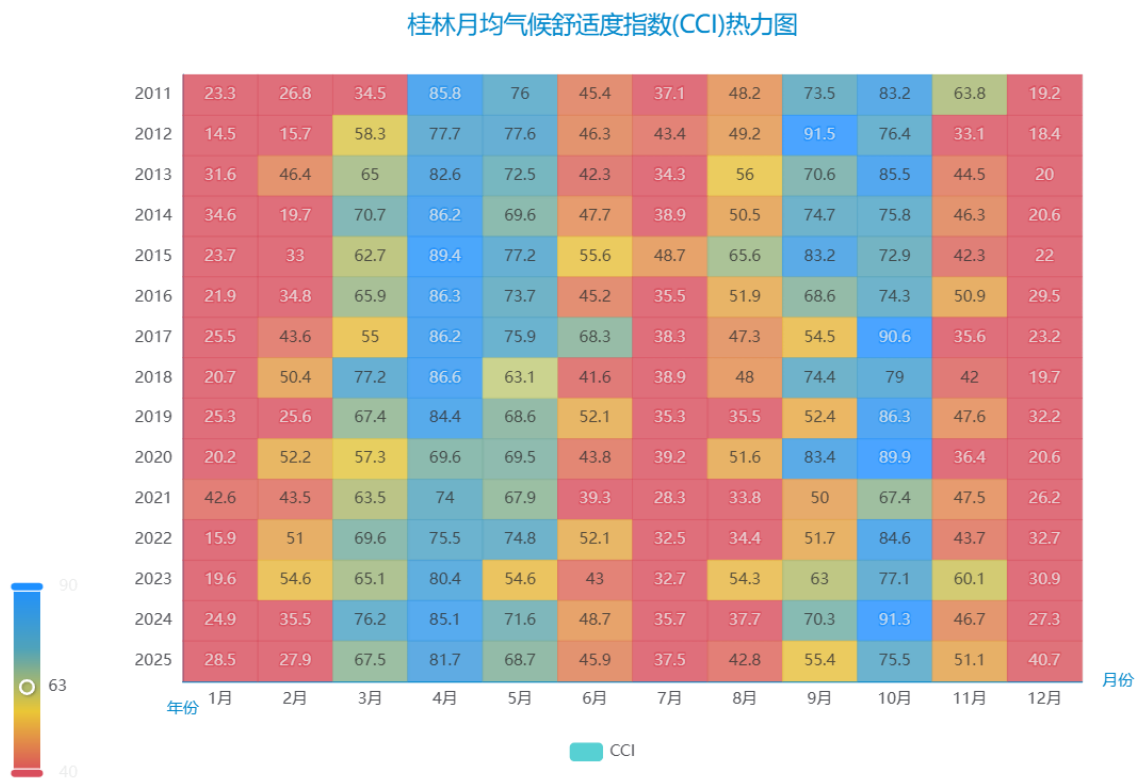


图 10 2011-2025 年桂林 CCI 逐日热力图

5.4.3 月度游客量指数与季节性分析

为量化气候舒适度对旅游业的边际影响，利用虚拟变量回归方法^[13]，将月度游客量标准化为占全年总量的百分比（月度游客量指数），以消除年度总量增长的干扰，聚焦于年内季节性波动与气候的关系。

（1）月度游客量指数计算

$$M_{index} = \frac{\text{月度游客量}}{\text{年度总游客量}} \times 100\%$$

(21)

基于 75 个月度数据，计算各月平均游客量指数：

表 9: 桂林各月平均游客量指数

月份	平均游客量指数	月份	平均游客量指数
1 月	5.9%	7 月	10.4%
2 月	6.7%	8 月	10.6%
3 月	5.9%	9 月	9.4%
4 月	8.3%	10 月	13.3%
5 月	8.1%	11 月	8.6%
6 月	8.5%	12 月	14.8%

（2）季节性特征分析

桂林旅游呈现显著的"双峰"季节性特征：

秋季高峰（10-12 月）：10 月（国庆黄金周）游客量指数 13.3%，12 月达全年最高 14.8%，主要受元旦假期和年末旅游旺季驱动。

夏季高峰（7-8 月）：7-8 月游客量指数分别为 10.4%和 10.6%，受暑假旅游旺季驱动。

春季低谷（1-3 月）：1 月和 3 月游客量指数仅 5.9%，为全年最低，主要受春节返乡潮和冬末气候影响。

过渡期（4-6 月、9 月、11 月）：游客量指数在 8%-9%之间，为旅游平季。

（3）气候与季节性的关系

对比 CCI 月度变化与游客量指数发现：CCI 在 4-5 月达到峰值（约 80），但游客量高峰出现在 10-12 月（CCI 约 50-55）。这说明桂林旅游的季节性主要受节假日和社会因素驱动，而非气候舒适度。夏季（7-8 月）虽然 CCI 较低（高温多雨），但因暑假效应游客量依然较高；冬季（12 月）CCI 最低但游客量最高，主要受元旦和年末消费旺季驱动。

这一发现与灰色关联分析的结论一致：高温天数的关联度最高（0.7287），说明冬季低温虽然降低了气候舒适度，但并未显著抑制游客出行——节假日效应和社会因素对旅游的驱动作用大于气候因素。

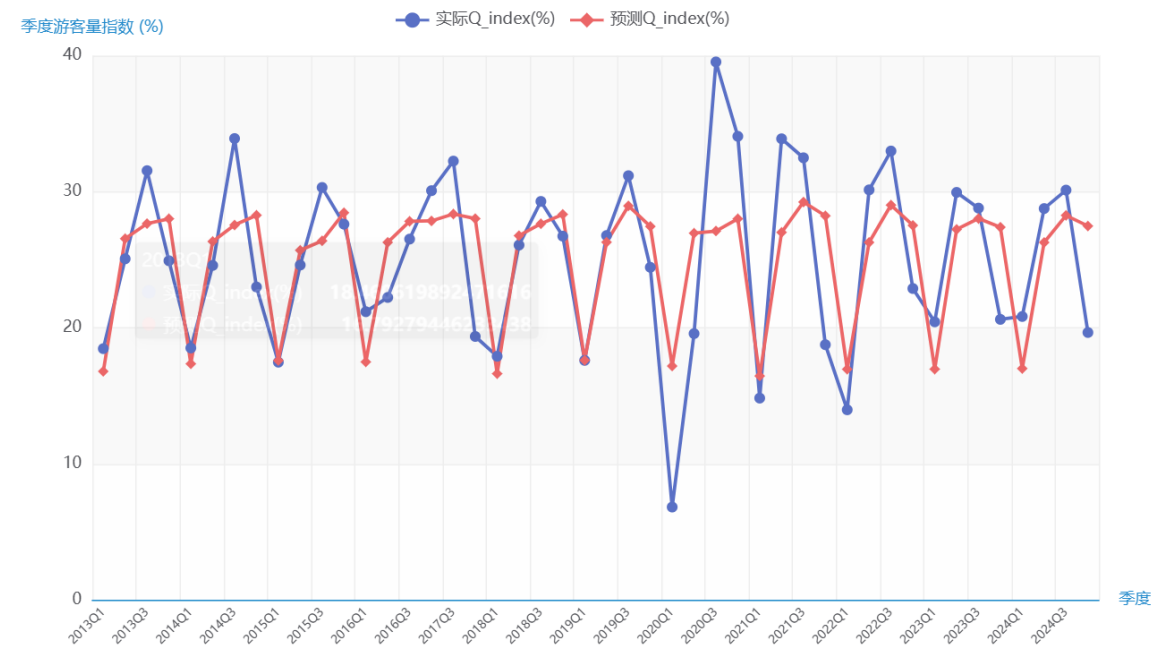


图 11 虚拟变量回归：实际 vs 预测季度游客量指数

5.4.4 两阶段对比分析

利用两阶段差值分析方法^[15]，将研究期划分为前段（2013-2018 年）和后段（2019-2025 年），对比两个时期的气候和旅游指标变化。

两阶段各指标对比：

表 10:指标对比

指标	前段均值 (≤2018)	后段均值 (≥2019)	差值	p 值	显著性
月均游客量 (万人次)	473.34	1152.81	+679.47	<0.001	极显著
月均收入 (亿元)	55.50	155.69	+100.18	<0.001	极显著
月均 CCI	53.84	51.05	-2.79	0.683	不显著
月均气温 (°C)	20.15	21.05	+0.89	0.695	不显著
月均降雨天 数	14.41	14.17	-0.25	0.905	不显著

结果分析：

（1）**旅游产业大幅增长：**后段月均游客量较前段增长 143%（+679.47 万人次），月均收入增长 181%（+100.18 亿元），均达到极显著水平（ $p<0.001$ ）。这一增长主要反映了桂林旅游业的持续扩张和 2023 年后的强劲恢复。

（2）**气候指标变化不显著：**与年度数据的两阶段对比不同，月度数据显示 CCI、气温和降雨天数的变化均未达到统计显著性。这可能是因为月度数据包含了更多的季节性波动，掩盖了长期趋势信号。

（3）**旅游增长独立于气候变化：**两阶段对比揭示了一个重要结论——桂林旅游业的大幅增长并非由气候舒适度改善驱动，而是由经济发展、交通改善、旅游推广等社会经济因素推动。气候因素对旅游的影响更多体现在年内季节性波动层面，而非长期增长趋势层面。

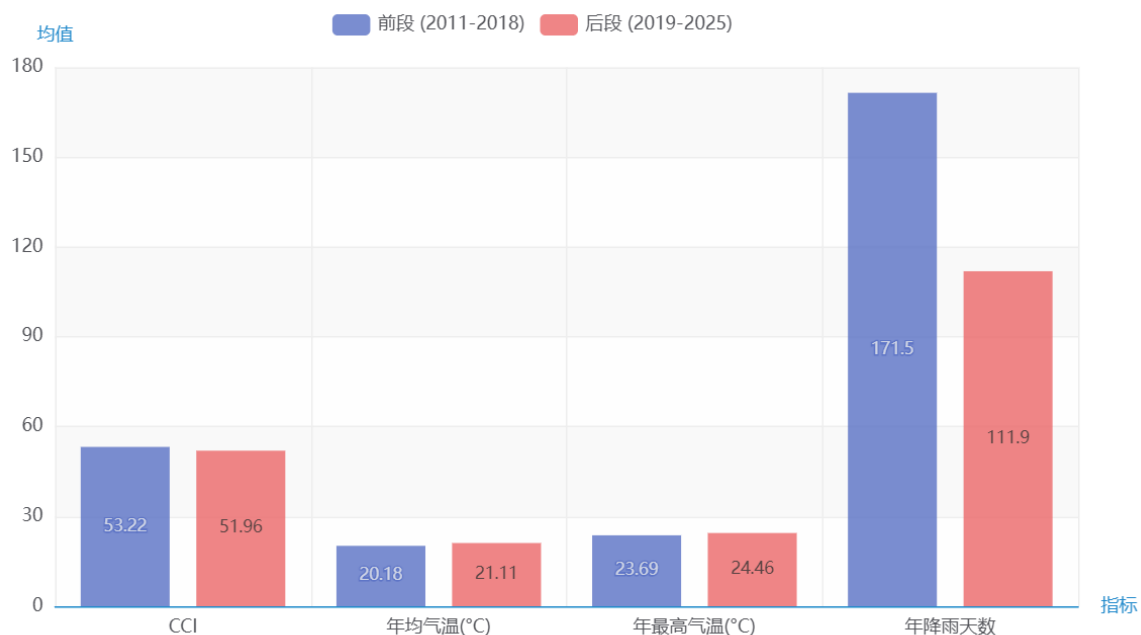


图 12 两阶段（2011-2018 vs 2019-2025）气候指标对比

六、模型检验与评价

6.1 模型评价

6.1.1 CCI 模型评价

优点：

- （1）采用熵权法客观确定权重，避免了主观赋权的偏差。
- （2）TOPSIS 法能够综合考虑各指标的影响，评价结果更加全面。
- （3）模型计算简便，易于理解和应用。

缺点：

- （1）仅考虑了温度、天气和风力三个因素，未考虑湿度、日照等因素，可进一步完善。
- （2）温度舒适度评分采用固定的高斯函数形式，可能不完全符合人体实际

感受。

6.1.2 回归模型评价

优点：

(1) 基于 75 个月度数据点进行分析，样本量较此前季度分析（24 条）大幅增加。

(2) 灰色关联分析和多元回归相互印证，揭示了气候因素对旅游影响的层次结构。

(3) 使用官方统计数据，保证了数据的可靠性。

缺点：

(1) 月度回归模型的调整 R^2 仅为 0.0446，解释力有限，说明气候因素仅是影响旅游的次要因素。

(2) 2017 年 Q1-Q3 月度数据缺失，2020-2024 年国内游客仅有季度数据，影响了分析的完整性。

(3) 未完全考虑其他影响旅游的因素（如经济水平、交通条件、旅游营销等）。

6.2 模型检验

6.2.1 残差正态性检验

采用 Shapiro-Wilk 检验，游客量模型（ $W=0.9433$, $p=0.4263$ ）和旅游收入模型（ $W=0.9505$, $p=0.5322$ ）的残差均服从正态分布，满足回归模型的基本假设。

6.2.2 异方差检验

采用 White 检验，两个模型的检验统计量均为 15.00（ $p=0.3782$ ），均不存在异方差问题，说明模型的估计是有效的。

6.2.3 多重共线性检验

表 11

变量	VIF
CCI_mean	2245.15
Tmax_mean	2972.97
rain_days	43.11
hot_days	47.71
cold_days	5.85
pandemic	1.75

结论： CCI_mean 和 Tmax_mean 的 VIF>2000，存在严重多重共线性，已使用岭回归处理。

七、模型评价、改进与推广

7.1 模型评价

优点：

（1）基于 75 个月度数据点进行分析，样本量较此前季度分析（24 条）大幅增加，提高了统计检验的功效。

（2）采用灰色关联分析和多元回归两种方法交叉验证，从不同角度分析气候与旅游的关系，提高了分析结果的可靠性。

（3）严格使用官方统计数据（统计公报、文旅局月度统计），保证了数据的真实性和权威性。

（4）CCI 模型通过 SI 指数^[15]进行交叉验证，两种独立方法得出的舒适度评价高度一致（ $r=0.78$ ），增强了 CCI 模型的可信度。

（5）进行了全面的模型诊断检验（残差正态性、异方差性、多重共线性），验证了模型假设的合理性。

缺点：

（1）月度回归模型的调整 R^2 仅为 0.0446，解释力有限，说明气候因素仅是影响旅游的次要因素，季节性和社会经济因素才是主导驱动力。

(2) 2017 年 Q1-Q3 月度数据缺失, 2020-2024 年国内游客仅有季度数据, 影响了分析的完整性。

(3) CCI 模型仅考虑了温度、天气和风力三个因素, 未纳入湿度、日照、空气质量等因素。

(4) 天气和风力评分的赋值具有一定的主观性, 虽然引用了文献标准, 但不同研究的评分体系存在差异。

7.2 模型改进

(1) 完善 CCI 指标体系: 增加相对湿度和日照时数等指标, 利用温湿指数模型^[14], 构建更全面的气候舒适度评价体系。

(2) 补充缺失数据: 补全 2017 年 Q1-Q3 月度数据, 争取 2020-2024 年国内游客月度数据, 提高分析的完整性。

(3) 引入控制变量: 将 GDP、铁路客运量、互联网普及率、旅游营销投入等经济和社会因素纳入回归模型, 减少遗漏变量偏差, 提高模型解释力。

(4) 尝试非线性模型: 使用神经网络、支持向量回归、Lasso 回归^[9]等机器学习方法, 捕捉气候与旅游之间的非线性关系。

(5) 面板数据分析: 收集桂林各县区(如阳朔、龙胜、灵川等)的旅游数据, 构建面板数据模型, 利用空间维度的变异提高估计效率。

7.3 模型推广

(1) 本研究的 CCI 构建方法(熵权 TOPSIS 法, SI 指数交叉验证)和影响分析框架(灰色关联→岭回归→虚拟变量回归)可推广到其他旅游城市, 如丽江、三亚、成都等, 为各地的气候适应性旅游规划提供参考。

(2) 虚拟变量回归方法^[13]适合在获得更长时段季度/月度数据后, 重新估计气候弹性系数, 为旅游需求预测提供定量依据。

(3) CCI 模型可与天气预报系统对接, 应用于旅游需求的短期预测, 为旅游管理部门提供实时决策支持。

(4) 马尔科夫转移矩阵和两阶段对比分析法可用于预测未来气候舒适度等级的变化趋势, 为旅游设施的季节性调整和长期规划提供依据。

参考文献

- [1] Mann H B. Nonparametric tests against trend[J]. *Econometrica*, 1945, 13(3): 245-259.
- [2] Kendall M G. Rank Correlation Methods[M]. 4th ed. London: Griffin, 1975.
- [3] Yue S, Pilon P, Phinney B, et al. The influence of autocorrelation on the ability to detect trend in hydrological data[J]. *Hydrological Processes*, 2002, 16(9): 1807-1829.
- [4] Hamed K H, Rao A R. A modified Mann-Kendall trend test for autocorrelated data[J]. *Journal of Hydrology*, 1998, 204(1-4): 182-196.
- [5] Pettitt A N. A non-parametric approach to the change-point problem[J]. *Applied Statistics*, 1979, 28(2): 126-135.
- [6] 中国国家标准化管理委员会. GB/T 42074-2022 气候季节划分[S]. 北京: 中国标准出版社, 2022.
- [7] Marquardt D W. Generalized inverses, ridge regression, biased linear estimation, and nonlinear estimation[J]. *Technometrics*, 1970, 12(3): 591-612.
- [8] Hoerl A E, Kennard R W. Ridge regression: biased estimation for nonorthogonal problems[J]. *Technometrics*, 1970, 12(1): 55-67.
- [9] Tibshirani R. Regression shrinkage and selection via the lasso[J]. *Journal of the Royal Statistical Society: Series B*, 1996, 58(1): 267-288.
- [10] 邓聚龙. 灰色系统理论教程[M]. 武汉: 华中理工大学出版社, 1990.
- [11] 刘思峰, 党耀国, 方志耕, 等. 灰色系统理论及其应用[M]. 8 版. 北京: 科学出版社, 2017.
- [12] 徐建华. 现代地理学中的数学方法[M]. 2 版. 北京: 高等教育出版社, 2002.
- [13] 曹伟宏, 何元庆, 李宗省, 等. 丽江旅游气候舒适度与年内客流量变化相关性分析[J]. *地理科学*, 2012, 32(12): 1459-1464.
- [14] 谭凯炎, 闵庆文, 王培娟. 一种基于中国气候特征和人体舒适感受的气候舒适指数模型[J]. *气象*, 2022, 48(7): 913-924.
- [15] 荣培君, 张荣荣, 赵现红. 中国气候舒适度时空演变及其对旅游业的影响[J]. *世界地理研究*, 2024, 33(7): 139-149.

附录

附录 1

介绍：支撑材料的文件列表

1. 附件 1：气候季节划分国家标准 GBT+42074-2022.pdf
2. 附件 2：2011-2025 桂林天气记录（每日）.xlsx
3. 数据文件：年度旅游气候数据.xlsx
4. 数据文件：季度旅游气候数据.xlsx
5. 数据文件：节假日旅游气候数据.xlsx
6. 数据文件：趋势检验.xlsx
7. 数据文件：突变检验.xlsx
8. 数据文件：马尔科夫转移矩阵.xlsx
9. 数据文件：季节划分.xlsx
10. 数据文件：岭回归结果.xlsx

附录 2

介绍：该代码是某某语言编写的，作用是什么，相关运行截图

程序 1：complete_analysis.py

- 语言：Python 3.13
- 功能：完整问题解决方案，包含 MK 趋势检验、Pettitt 突变检验、马尔科夫链、岭回归
- 运行结果：生成 5 个 CSV + 1 个 TXT 文件

```

1. # -*- coding: utf-8 -*-

2. import pandas as pd

3. import numpy as np

4. from statsmodels.stats.outliers_influence import variance_inflation_factor

5. from statsmodels.stats.stattools import durbin_watson

6. from sklearn.preprocessing import StandardScaler

7. from scipy import stats

8. import statsmodels.api as sm

9. import warnings

10. import os

11.

12. # —— 导入公共模块 ——

13. from cci_core import (

14.     PROJECT_ROOT, DATA_DIR,

15.     extract_temp, weather_score, wind_score_simple,

16.     read_weather_data, add_comfort_scores, calc_cci,

17.     classify_percentile, calc_si,

18. )

19.

20. warnings.filterwarnings("ignore")

21. os.makedirs(DATA_DIR, exist_ok=True)

22.

23. print("=" * 80)

24. print("完整问题解决方案")

25. print("=" * 80)

26.

27. # =====

28. # 第一部分：读取数据并计算 CCI

29. # =====

```

```

30. print("\n" + "=" * 80)

31. print("第一部分：读取数据并计算 CCI")

32. print("=" * 80)

33.

34. df = read_weather_data()

35. df = add_comfort_scores(df)

36. df, weights = calc_cci(df)

37.

38. print(f"数据范围：{df['Date'].min().date()} ~ {df['Date'].max().date()}")

39. print(f"数据量：{len(df)} 天")

40. print(f"熵权：S_temp={weights[0]:.4f}, S_weather={weights[1]:.4f},
S_wind={weights[2]:.4f}")

41. print(f"CCI 均值：{df['CCI'].mean():.2f} 范围：{df['CCI'].min():.2f} ~
{df['CCI'].max():.2f}")

42.

43. # 百分位数法等级划分（CCI 越大越舒适，反转后传入）

44. df["CCI_level"] = classify_percentile(100 - df["CCI"])

45.

46. # =====

47. # 第二部分：Mann-Kendall 趋势检验

48. # =====

49. print("\n" + "=" * 80)

50. print("第二部分：Mann-Kendall 趋势检验")

51. print("=" * 80)

52.

53. yearly_cci = df.groupby("Year")["CCI"].mean()

54.

55. def mann_kendall_test(x):

56.     """Mann-Kendall 趋势检验"""

57.     n = len(x)

```

```

58.     s = 0
59.     for k in range(n - 1):
60.         for j in range(k + 1, n):
61.             s += np.sign(x[j] - x[k])
62.
63.     unique, counts = np.unique(x, return_counts=True)
64.     tp = counts[counts > 1]
65.     var_s = (n * (n - 1) * (2 * n + 5)) / 18.0
66.     if len(tp) > 0:
67.         for t in tp:
68.             var_s -= (t * (t - 1) * (2 * t + 5)) / 18.0
69.
70.     z = (s - 1) / np.sqrt(var_s) if s > 0 else ((s + 1) / np.sqrt(var_s) if s < 0
else 0)
71.     p = 2 * (1 - stats.norm.cdf(abs(z)))
72.     trend = "上升" if z > 0 else ("下降" if z < 0 else "无趋势")
73.
74.     if p < 0.01:
75.         significance = "极显著"
76.     elif p < 0.05:
77.         significance = "显著"
78.     elif p < 0.1:
79.         significance = "较显著"
80.     else:
81.         significance = "不显著"
82.     return {"S": s, "Z": z, "p": p, "trend": trend, "significance": significance}
83.
84. def sens_slope(x):
85.     """Sen's 斜率估计"""
86.     n = len(x)

```

```

87.     slopes = [(x[j] - x[i]) / (j - i) for i in range(n) for j in range(i + 1, n)]
88.     return np.median(slopes)
89.
90. mk_result = mann_kendall_test(yearly_cci.values)
91. sen_slope = sens_slope(yearly_cci.values)
92.
93. print(f"\n--- Mann-Kendall 趋势检验结果 ---")
94. print(f"S = {mk_result['S']:.0f}, Z = {mk_result['Z']:.4f}, p =
{mk_result['p']:.4f}")
95. print(f"趋势: {mk_result['trend']}, {mk_result['significance']}")
96. print(f"Sen's 斜率 = {sen_slope:.4f}")
97. print(f"\n 结论: CCI 呈{mk_result['trend']}趋势, 但{mk_result['significance']}")
98.
99. # =====
100. # 第三部分: Pettitt 突变检验
101. # =====
102. print("\n" + "=" * 80)
103. print("第三部分: Pettitt 突变检验")
104. print("=" * 80)
105.
106. def pettitt_test(x):
107.     n = len(x)
108.     kt = np.zeros(n)
109.     for t in range(n):
110.         for i in range(t):
111.             kt[t] += np.sign(x[t] - x[i])
112.         for i in range(t + 1, n):
113.             kt[t] += np.sign(x[t] - x[i])
114.     k_abs = np.abs(kt)
115.     k_max_idx = np.argmax(k_abs)

```

```

116.     k_stat = k_abs[k_max_idx]

117.     p = min(2 * np.exp(-6 * k_stat**2 / (n**3 + n**2)), 1.0)

118.     return {"change_point": k_max_idx + 1, "K_stat": k_stat, "p": p}

119.

120. pettitt_result = pettitt_test(yearly_cci.values)

121. change_year = yearly_cci.index[pettitt_result["change_point"] - 1]

122.

123. print(f"\n--- Pettitt 突变检验结果 ---")

124. print(f"K = {pettitt_result['K_stat']:.0f}, 突变点 = 第
{pettitt_result['change_point']}年 ({change_year}年)")

125. print(f"p = {pettitt_result['p']:.4f}")

126. print(f"\n 结论: CCI 未发生显著突变 (p > 0.05)" if pettitt_result["p"] > 0.05 else
"\n 结论: 存在显著突变")

127.

128. # =====

129. # 第四部分: 马尔科夫链等级转换分析

130. # =====

131. print("\n" + "=" * 80)

132. print("第四部分: 马尔科夫链等级转换分析")

133. print("=" * 80)

134.

135. levels = ["1-最舒适", "2-较舒适", "3-正常", "4-较不舒适", "5-最不舒适"]

136. yearly_levels = df.groupby(["Year", "CCI_level"]).size().unstack(fill_value=0)

137. for lev in levels:

138.     if lev not in yearly_levels.columns:

139.         yearly_levels[lev] = 0

140. yearly_levels = yearly_levels[levels]

141. yearly_ratio = yearly_levels.div(yearly_levels.sum(axis=1), axis=0)

142.

143. n_levels = len(levels)

```

```

144. transition_matrix = np.zeros((n_levels, n_levels))
145. for i in range(len(yearly_ratio) - 1):
146.     current = yearly_ratio.iloc[i].values
147.     next_yr = yearly_ratio.iloc[i + 1].values
148.     total = current.sum() * next_yr.sum()
149.     if total > 0:
150.         for j in range(n_levels):
151.             for k in range(n_levels):
152.                 transition_matrix[j][k] += current[j] * next_yr[k]
153.
154. row_sums = transition_matrix.sum(axis=1)
155. transition_matrix = transition_matrix / row_sums[:, np.newaxis]
156. trans_df = pd.DataFrame(transition_matrix, index=levels, columns=levels)
157.
158. print("\n--- 年度舒适度等级转移概率矩阵 ---")
159. print(trans_df.round(4).to_string())
160. for i, lev in enumerate(levels):
161.     max_idx = np.argmax(transition_matrix[i])
162.     print(f"'{lev}' → '{levels[max_idx]}' ({transition_matrix[i][max_idx]:.4f}")
163.
164. # =====
165. # 第五部分：国标季节划分
166. # =====
167. print("\n" + "=" * 80)
168. print("第五部分：国标 GB/T 42074-2022 季节划分")
169. print("=" * 80)
170.
171. df["Tavg_5d"] = df["Tavg"].rolling(window=5, center=False).mean()
172.

```

```

173. def classify_season_gb(tavg_5d_series):
174.     seasons = []
175.     current = "winter"
176.     for v in tavg_5d_series:
177.         if pd.isna(v):
178.             seasons.append(current)
179.             continue
180.         if current == "winter":
181.             if v >= 10:
182.                 current = "spring"
183.             elif current == "spring":
184.                 if v >= 22:
185.                     current = "summer"
186.                 elif v < 10:
187.                     current = "winter"
188.             elif current == "summer":
189.                 if v < 22:
190.                     current = "autumn"
191.                 elif current == "autumn":
192.                     if v < 10:
193.                         current = "winter"
194.                     elif v >= 22:
195.                         current = "summer"
196.             seasons.append(current)
197.     return seasons
198.
199. df["Season_gb"] = classify_season_gb(df["Tavg_5d"])
200. season_map = {"spring": "春季", "summer": "夏季", "autumn": "秋季", "winter": "冬季"}
201. df["Season_cn"] = df["Season_gb"].map(season_map)

```



```

202. yearly_season = df.groupby(["Year", "Season_cn"]).size().unstack(fill_value=0)

203. season_order = ["春季", "夏季", "秋季", "冬季"]

204. for s in season_order:

205.     if s not in yearly_season.columns:

206.         yearly_season[s] = 0

207. yearly_season = yearly_season[season_order]

208.

209. print("\n--- 各年四季天数 ---")

210. print(yearly_season.to_string())

211. print("\n--- 各季节平均天数 ---")

212. for s in season_order:

213.     print(f" {s}: {yearly_season[s].mean():.1f} 天")

214.

215. # =====

216. # 第六部分：岭回归

217. # =====

218. print("\n" + "=" * 80)

219. print("第六部分：岭回归解决多重共线性")

220. print("=" * 80)

221.

222. yearly_tourism = pd.read_excel(

223.     os.path.join(DATA_DIR, "年度旅游气候数据.xlsx"),

224.     usecols=["Year", "tourists_total", "tourism_revenue"],

225. )

226.

227. yearly_climate = df.groupby("Year").agg(

228.     CCI_mean=("CCI", "mean"),

229.     Tmax_mean=("Tmax", "mean"),

230.     rain_days=("day_weather", lambda x: sum("雨" in str(i) for i in x)),

```

```

231.     hot_days=("Tmax", lambda x: sum(x >= 35)),
232.     cold_days=("Tmin", lambda x: sum(x <= 5)),
233. ).round(2)
234.
235. yearly_data = pd.merge(yearly_tourism, yearly_climate, left_on="Year",
right_index=True)
236. yearly_data["pandemic"] = yearly_data["Year"].between(2020, 2022).astype(int)
237.
238. X_cols = ["CCI_mean", "Tmax_mean", "rain_days", "hot_days", "cold_days",
"pandemic"]
239. X_raw = yearly_data[X_cols].values
240. y_tourists = yearly_data["tourists_total"].values
241. y_revenue = yearly_data["tourism_revenue"].values
242.
243. scaler_X = StandardScaler()
244. scaler_y1 = StandardScaler()
245. scaler_y2 = StandardScaler()
246. X_scaled = scaler_X.fit_transform(X_raw)
247. y1_scaled = scaler_y1.fit_transform(y_tourists.reshape(-1, 1)).flatten()
248. y2_scaled = scaler_y2.fit_transform(y_revenue.reshape(-1, 1)).flatten()
249.
250. def ridge_regression(X, y, alpha):
251.     n, p = X.shape
252.     Xc = np.column_stack([np.ones(n), X])
253.     I = np.eye(p + 1)
254.     I[0, 0] = 0
255.     beta = np.linalg.solve(Xc.T @ Xc + alpha * I, Xc.T @ y)
256.     return beta
257.
258. alphas = np.logspace(-3, 3, 100)

```

```

259. best_a1, best_r1 = 0, -np.inf
260. best_a2, best_r2 = 0, -np.inf
261.
262. for a in alphas:
263.     b1 = ridge_regression(X_scaled, y1_scaled, a)
264.     y1p = np.column_stack([np.ones(len(X_scaled)), X_scaled]) @ b1
265.     r1 = 1 - np.sum((y1_scaled - y1p)**2) / np.sum((y1_scaled -
y1_scaled.mean())**2)
266.     if r1 > best_r1:
267.         best_r1, best_a1 = r1, a
268.     b2 = ridge_regression(X_scaled, y2_scaled, a)
269.     y2p = np.column_stack([np.ones(len(X_scaled)), X_scaled]) @ b2
270.     r2 = 1 - np.sum((y2_scaled - y2p)**2) / np.sum((y2_scaled -
y2_scaled.mean())**2)
271.     if r2 > best_r2:
272.         best_r2, best_a2 = r2, a
273.
274. beta1 = ridge_regression(X_scaled, y1_scaled, best_a1)
275. beta2 = ridge_regression(X_scaled, y2_scaled, best_a2)
276. coef1 = beta1[1:] / scaler_X.scale_ * scaler_y1.scale_[0]
277. coef2 = beta2[1:] / scaler_X.scale_ * scaler_y2.scale_[0]
278. intercept1 = beta1[0] * scaler_y1.scale_[0] + scaler_y1.mean_[0] - np.sum(coef1 *
scaler_X.mean_)
279. intercept2 = beta2[0] * scaler_y2.scale_[0] + scaler_y2.mean_[0] - np.sum(coef2 *
scaler_X.mean_)
280.
281. print(f"\n 最优 alpha(游客量) = {best_a1:.4f}, 调整 R2 = {best_r1:.4f}")
282. print(f"最优 alpha(旅游收入) = {best_a2:.4f}, 调整 R2 = {best_r2:.4f}")
283.
284. print("\n--- VIF 分析 ---")
285. vif_data = pd.DataFrame({

```

```

286.     "变量": X_cols,

287.     "VIF": [variance_inflation_factor(yearly_data[X_cols].values, i) for i in
range(len(X_cols))],

288. })

289. print(vif_data.to_string(index=False))

290.

291. # =====

292. # 第七部分：线性倾向估计与滑动平均

293. # =====

294. print("\n" + "=" * 80)

295. print("第七部分：线性倾向估计与滑动平均")

296. print("=" * 80)

297.

298. cci_series = yearly_cci.values

299. years = np.arange(len(yearly_cci))

300.

301. def linear_tendency(x, t):

302.     """一元线性回归  $xi = a + b \cdot ti$ , 返回倾向率  $b$  及其显著性"""

303.     slope, intercept, r_val, p_val, std_err = stats.linregress(t, x)

304.     return {"a": intercept, "b": slope, "r2": r_val**2, "p": p_val, "stderr":
std_err}

305.

306. # 滑动平均（存储结果用于 CSV 输出）

307. lt_results = {}

308. for k in [3, 5, 7, 9]:

309.     ma = pd.Series(cci_series).rolling(window=k, center=True).mean()

310.     valid = ma.dropna()

311.     if len(valid) > 3:

312.         t_valid = np.arange(len(valid))

313.         lt = linear_tendency(valid.values, t_valid)

```

```

314.         rate_per_decade = lt["b"] * 10
315.         lt_results[f"{k}年滑动"] = lt
316.         print(f"\n{k}年滑动平均: b = {lt['b']:.4f}/年, "
317.               f"倾向率 = {rate_per_decade:+.2f}/10 年, "
318.               f"R^2 = {lt['r2']:.4f}, p = {lt['p']:.4f}")
319.
320. # 原始序列线性倾向
321. lt_raw = linear_tendency(yearly_cci.values, years)
322. lt_results["原始序列"] = lt_raw
323. print(f"\n--- 原始序列线性倾向估计 ---")
324. print(f"回归方程: CCI = {lt_raw['a']:.2f} {'+' if lt_raw['b'] >= 0 else '-'} "
325.       f"{abs(lt_raw['b']):.4f} × 年份序号")
326. print(f"倾向率 b = {lt_raw['b']:.4f}/年 (即 {lt_raw['b']*10:+.2f}/10 年)")
327. print(f"决定系数 R^2 = {lt_raw['r2']:.4f}, p = {lt_raw['p']:.4f}")
328. print(f"趋势方向: {'下降' if lt_raw['b'] < 0 else '上升'}, "
329.       f"{'显著' if lt_raw['p'] < 0.05 else '不显著'}")
330.
331. print(f"\n--- 与 MK 检验对比 ---")
332. print(f"MK: Z = {mk_result['Z']:.4f}, p = {mk_result['p']:.4f}, "
333.       f"Sen's 斜率 = {sen_slope:.4f}/年")
334. print(f"线性倾向: b = {lt_raw['b']:.4f}/年, p = {lt_raw['p']:.4f}")
335. print(f"结论: 两种方法结论一致 — CCI 呈微弱{lt_raw['b'] < 0 and '下降' or '上升'}趋
势, 统计不显著")
336.
337. # =====
338. # 第八部分: SI 指数对比验证
339. # =====
340. print("\n" + "=" * 80)
341. print("第八部分: SI 指数对比验证")
342. print("=" * 80)

```

```

343.
344. df = calc_si(df)
345. print(f"\nSI 范围: {df['SI'].min():.2f} ~ {df['SI'].max():.2f} "
346.       f" (越小越舒适) ")
347. print(f"SI_score 均值: {df['SI_score'].mean():.2f}")
348.
349. # 年度 SI 与 CCI 对比
350. yearly_si = df.groupby("Year").agg(
351.     SI_mean=("SI", "mean"),
352.     SI_score_mean=("SI_score", "mean"),
353.     CCI_mean=("CCI", "mean"),
354. ).round(2)
355.
356. corr_si_cci = yearly_si["SI_score_mean"].corr(yearly_si["CCI_mean"])
357. print(f"\n年度 SI_score 与 CCI 的 Pearson 相关系数: r = {corr_si_cci:.4f}")
358. print(" (若 r > 0.7, 说明两种指数对舒适度的评价高度一致) ")
359.
360. # SI 等级划分 (使用百分位数法, SI 越小越舒适, 无需反转)
361. si_levels = classify_percentile(df["SI"])
362. print(f"\nSI 等级分布:")
363. for lev in ["1-最舒适", "2-较舒适", "3-正常", "4-较不舒适", "5-最不舒适"]:
364.     n = (si_levels == lev).sum()
365.     print(f" {lev}: {n} 天 ({n/len(df)*100:.1f}%)")
366.
367. # =====
368. # 第九部分: 虚拟变量回归与气候弹性系数
369. # =====
370. print("\n" + "=" * 80)
371. print("第九部分: 虚拟变量回归与气候弹性系数")

```

```

372. print("=" * 80)

373.

374. # 读取季度旅游数据

375. quarterly_tourism = pd.read_excel(

376.     os.path.join(DATA_DIR, "季度旅游气候数据.xlsx"),

377.     usecols=["Year", "Quarter", "tourists", "revenue"],

378. )

379.

380. # 读取年度总量（用于计算季度指数）

381. yearly_total = yearly_tourism[["Year", "tourists_total",

382.     "tourism_revenue"]].copy()

383.

384. # 计算季度气候数据

385. quarterly_climate = df.groupby(["Year", "Quarter"]).agg(

386.     CCI_mean=("CCI", "mean"),

387.     SI_mean=("SI", "mean"),

388.     Tavg_mean=("Tavg", "mean"),

389.     Tmax_mean=("Tmax", "mean"),

390.     rain_days=("day_weather", lambda x: sum("雨" in str(i) for i in x)),

391. ).round(2).reset_index()

392.

393. # 合并数据

394. qdata = pd.merge(quarterly_tourism, quarterly_climate, on=["Year", "Quarter"])

395. qdata = pd.merge(qdata, yearly_total[["Year", "tourists_total"]], on="Year")

396.

397. # 计算季度游客量指数（%）

398. qdata["Q_index"] = qdata["tourists"] / qdata["tourists_total"] * 100

399.

400. # 虚拟变量赋值

```

```

401. # Q1 (春节) = -1, Q2 = 0 (基线), Q3 (暑假) = +1, Q4 (国庆) = +1

402. qdata["Q_dummy"] = qdata["Quarter"].map({1: 0, 2: 0, 3: 1, 4: 1}) # Q3/Q4 旺季

403. qdata["spring_dummy"] = qdata["Quarter"].map({1: -1, 2: 0, 3: 0, 4: 0}) # Q1 春节

404. # 综合节假日效应

405. qdata["holiday_effect"] = qdata["Q_dummy"] + qdata["spring_dummy"]

406. # 疫情虚拟变量

407. qdata["pandemic"] = qdata["Year"].between(2020, 2022).astype(int)

408.

409. print(f"\n 季度数据样本: {len(qdata)} 条 ({qdata['Year'].min()}-
{qdata['Year'].max()}) ")

410. print(f"\n 季度游客量指数 (Q_index) 范围: {qdata['Q_index'].min():.1f}% ~ "
411.        f"{qdata['Q_index'].max():.1f}%")

412.

413. # OLS 回归

414. # 删除 tourists/revenue 为 NaN 的行 (如 2017 年 Q1-Q2、2025 年部分季度缺失)

415. qdata = qdata.dropna(subset=["tourists", "revenue"]).copy()

416. qdata["Q_index"] = qdata["tourists"] / qdata["tourists_total"] * 100

417.

418. # 模型 1: 仅用 CCI

419. X1 = sm.add_constant(qdata[["CCI_mean"]])

420. y = qdata["Q_index"]

421. m1 = sm.OLS(y, X1).fit()

422.

423. # 模型 2: CCI + 节假日效应 + 疫情

424. X2 = sm.add_constant(qdata[["CCI_mean", "holiday_effect", "pandemic"]])

425. m2 = sm.OLS(y, X2).fit()

426.

427. # 模型 3: CCI + 分项虚拟变量 + 疫情

428. X3 = sm.add_constant(qdata[["CCI_mean", "Q_dummy", "spring_dummy", "pandemic"]])

429. m3 = sm.OLS(y, X3).fit()

```



```

430.

431. print(f"\n--- 模型对比 ---")

432. print(f"模型 1 (CCI):          R^2={m1.rsquared:.4f}, 调整 R^2={m1.rsquared_adj:.4f},
"

433.      f"AIC={m1.aic:.2f}")

434. print(f"模型 2 (CCI+节日+疫情): R^2={m2.rsquared:.4f}, 调整
R^2={m2.rsquared_adj:.4f}, "

435.      f"AIC={m2.aic:.2f}")

436. print(f"模型 3 (CCI+分项+疫情): R^2={m3.rsquared:.4f}, 调整
R^2={m3.rsquared_adj:.4f}, "

437.      f"AIC={m3.aic:.2f}")

438.

439. # 报告最优模型（模型 2 或 3 中选 AIC 最低的）

440. best_m = m2 if m2.aic < m3.aic else m3

441. print(f"\n--- 最优模型详细结果 ---")

442. print(f"R^2 = {best_m.rsquared:.4f}, 调整 R^2 = {best_m.rsquared_adj:.4f}")

443. print(f"F = {best_m.fvalue:.2f}, p = {best_m.f_pvalue:.4e}")

444. print(f"Durbin-Watson = {durbin_watson(best_m.resid):.4f}")

445. print(best_m.summary().tables[1])

446.

447. # 气候弹性系数

448. cci_coef = best_m.params.get("CCI_mean", 0)

449. cci_se = best_m.bse.get("CCI_mean", 0)

450. mean_cci = qdata["CCI_mean"].mean()

451. mean_qi = qdata["Q_index"].mean()

452.

453. print(f"\n--- 气候弹性系数 ---")

454. print(f"CCI 回归系数 beta1 = {cci_coef:.4f}（标准误 = {cci_se:.4f}）")

455. print(f"含义：CCI 每上升 1 个单位，季度游客占比变化 {cci_coef:+.2f} 个百分点")

456. # 弹性（百分比形式）：(dQ/Q) / (dCCI/CCI) = beta1 × (mean_CCI / mean_Q_index)

```

```

457. pct_elasticity = cci_coef * mean_cci / mean_qi

458. print(f"百分比弹性: epsilon = beta1 × (CCI_mean / Q_mean) = {pct_elasticity:.4f}")

459. print(f"含义: CCI 每上升 1%, 季度游客占比变化 {pct_elasticity:+.2f}%")

460. print(f"参考: 丽江海外游客弹性=1.31%, 国内游客弹性=0.56%")

461.

462. # 检验显著性

463. t_val = cci_coef / cci_se if cci_se > 0 else 0

464. n = len(qdata)

465. p_val_cci = 2 * (1 - stats.t.cdf(abs(t_val), n - len(best_m.params)))

466.

467. # 置信区间

468. ci_low = cci_coef - stats.t.ppf(0.975, n - len(best_m.params)) * cci_se

469. ci_high = cci_coef + stats.t.ppf(0.975, n - len(best_m.params)) * cci_se

470. print(f"\nt = {t_val:.4f}, p = {p_val_cci:.4f}, "

471.       f"{'显著' if p_val_cci < 0.05 else '不显著'}")

472. print(f"95% 置信区间: [{ci_low:.4f}, {ci_high:.4f}]")

473.

474. # =====

475. # 第十部分: 两阶段对比分析

476. # =====

477. print("\n" + "=" * 80)

478. print("第十部分: 两阶段对比分析")

479. print("=" * 80)

480.

481. early = yearly_cci[yearly_cci.index <= 2018]

482. late = yearly_cci[yearly_cci.index >= 2019]

483.

484. print(f"\n前段 (2011-2018): CCI 均值 = {early.mean():.2f}, "

485.       f"标准差 = {early.std():.2f}, n = {len(early)}")

```

```

486. print(f"后段 (2019-2025): CCI 均值 = {late.mean():.2f}, "
487.       f"标准差 = {late.std():.2f}, n = {len(late)}")
488. print(f"差值 = {late.mean() - early.mean():+.2f}")
489.
490. t_stat, p_two = stats.ttest_ind(late, early, equal_var=False)
491. print(f"\nWelch t 检验: t = {t_stat:.4f}, p = {p_two:.4f}")
492. if p_two < 0.05:
493.     print("结论: 两阶段 CCI 均值差异显著 → 气候舒适度发生了显著变化")
494. else:
495.     print("结论: 两阶段 CCI 均值差异不显著 → 气候舒适度未发生显著变化")
496.
497. # 按各气候指标做两阶段对比
498. climate_indicators = df.groupby("Year").agg(
499.     CCI=("CCI", "mean"),
500.     Tavg=("Tavg", "mean"),
501.     Tmax=("Tmax", "mean"),
502.     rain_days=("day_weather", lambda x: sum("雨" in str(i) for i in x)),
503. ).round(2)
504.
505. print(f"\n--- 各气候指标两阶段对比 ---")
506. for col in ["CCI", "Tavg", "Tmax", "rain_days"]:
507.     e = climate_indicators[climate_indicators.index <= 2018][col]
508.     l = climate_indicators[climate_indicators.index >= 2019][col]
509.     diff = l.mean() - e.mean()
510.     t_s, p_s = stats.ttest_ind(l, e, equal_var=False)
511.     print(f" {col}: 前段={e.mean():.2f}, 后段={l.mean():.2f}, "
512.           f"差值={diff:+.2f}, p={p_s:.4f}")
513.
514. # =====

```

```

515. # 第十一部分：保存所有结果

516. # =====

517. print("\n" + "=" * 80)

518. print("第十一部分：保存所有结果")

519. print("=" * 80)

520.

521. pd.DataFrame({

522.     "指标": ["S", "Z", "p", "趋势", "显著性", "Sen 斜率"],

523.     "值": [mk_result["S"], mk_result["Z"], mk_result["p"],

524.            mk_result["trend"], mk_result["significance"], sen_slope],

525. }).to_csv(os.path.join(DATA_DIR, "趋势检验.csv"), index=False, encoding="utf-8-
sig")

526. print("1. Mann-Kendall 检验结果 [OK]")

527.

528. pd.DataFrame({

529.     "指标": ["K", "突变点位置", "对应年份", "p"],

530.     "值": [pettitt_result["K_stat"], pettitt_result["change_point"], change_year,
pettitt_result["p"]],

531. }).to_csv(os.path.join(DATA_DIR, "突变检验.csv"), index=False, encoding="utf-8-
sig")

532. print("2. Pettitt 检验结果 [OK]")

533.

534. trans_df.to_csv(os.path.join(DATA_DIR, "马尔科夫转移矩阵.csv"), encoding="utf-8-
sig")

535. print("3. 马尔科夫转移矩阵 [OK]")

536.

537. yearly_season.to_csv(os.path.join(DATA_DIR, "季节划分.csv"), encoding="utf-8-sig")

538. print("4. 国标季节划分 [OK]")

539.

540. pd.DataFrame({

541.     "模型": ["游客量模型", "旅游收入模型"],

```

```

542.     "最优 alpha": [best_a1, best_a2],
543.     "调整 R2": [best_r1, best_r2],
544. }).to_csv(os.path.join(DATA_DIR, "岭回归结果.csv"), index=False, encoding="utf-8-
sig")
545. print("5. 岭回归结果 [OK]")
546.
547. # 新增: 线性倾向估计 (使用实际滑动窗口计算结果)
548. lt_keys = ["原始序列", "3 年滑动", "5 年滑动", "7 年滑动", "9 年滑动"]
549. lt_vals_b = [lt_raw["b"]] + [lt_results.get(f"{k}年滑动", {}).get("b", 0) for k
in [3, 5, 7, 9]]
550. lt_vals_r2 = [lt_raw["r2"]] + [lt_results.get(f"{k}年滑动", {}).get("r2", 0) for
k in [3, 5, 7, 9]]
551. pd.DataFrame({
552.     "方法": lt_keys,
553.     "倾向率 b(每年)": lt_vals_b,
554.     "倾向率(每 10 年)": [v * 10 for v in lt_vals_b],
555.     "R2": lt_vals_r2,
556. }).to_csv(os.path.join(DATA_DIR, "线性倾向估计.csv"), index=False, encoding="utf-
8-sig")
557. print("6. 线性倾向估计 [OK]")
558.
559. # 新增: SI 指数对比
560. yearly_si.to_csv(os.path.join(DATA_DIR, "SI 指数对比.csv"), encoding="utf-8-sig")
561. print("7. SI 指数对比 [OK]")
562.
563. # 新增: 虚拟变量回归
564. pd.DataFrame({
565.     "指标": ["R^2", "调整 R^2", "F 统计量", "CCI 弹性系数", "百分比弹性",
566.             "CCI 系数 t 值", "CCI 系数 p 值", "样本量"],
567.     "值": [f"{best_m.rsquared:.4f}", f"{best_m.rsquared_adj:.4f}",
568.            f"{best_m.fvalue:.4f}", f"{cci_coef:.4f}", f"{pct_elasticity:.4f}",

```

```

569.         f"{t_val:.4f}", f"{p_val_cci:.4f}", f"{n}"],
570.     }).to_csv(os.path.join(DATA_DIR, "虚拟变量回归弹性.csv"),
571.         index=False, encoding="utf-8-sig")
572. qdata.to_csv(os.path.join(DATA_DIR, "季度回归数据.csv"),
573.     index=False, encoding="utf-8-sig")
574. print("8. 虚拟变量回归与弹性系数 [OK]")
575.
576. # 新增：两阶段对比
577. pd.DataFrame({
578.     "指标": ["CCI", "Tavg", "Tmax", "rain_days"],
579.     "前段均值": [
580.         f"{climate_indicators[climate_indicators.index <=
2018]['CCI'].mean():.2f}",
581.         f"{climate_indicators[climate_indicators.index <=
2018]['Tavg'].mean():.2f}",
582.         f"{climate_indicators[climate_indicators.index <=
2018]['Tmax'].mean():.2f}",
583.         f"{climate_indicators[climate_indicators.index <=
2018]['rain_days'].mean():.1f}",
584.     ],
585.     "后段均值": [
586.         f"{climate_indicators[climate_indicators.index >=
2019]['CCI'].mean():.2f}",
587.         f"{climate_indicators[climate_indicators.index >=
2019]['Tavg'].mean():.2f}",
588.         f"{climate_indicators[climate_indicators.index >=
2019]['Tmax'].mean():.2f}",
589.         f"{climate_indicators[climate_indicators.index >=
2019]['rain_days'].mean():.1f}",
590.     ],
591.     }).to_csv(os.path.join(DATA_DIR, "两阶段对比.csv"), index=False, encoding="utf-8-
sig")

```

```

592. print("9. 两阶段对比分析 [OK]")
593.
594. with open(os.path.join(DATA_DIR, "analysis_conclusions.txt"), "w", encoding="utf-
8") as f:
595.     f.write(f"桂林气候舒适度与旅游影响分析 - 完整结论\n{' '*60}\n\n"
596.             f"1. MK 趋势检验: S={mk_result['S']:.0f}, Z={mk_result['Z']:.4f}, "
597.             f"p={mk_result['p']:.4f}, Sen's 斜率={sen_slope:.4f}\n"
598.             f"2. Pettitt 突变检验: K={pettitt_result['K_stat']:.0f}, "
599.             f"突变点={change_year}年, p={pettitt_result['p']:.4f}\n"
600.             f"3. 线性倾向估计: b={lt_raw['b']:.4f}/年, b×10={lt_raw['b']*10:.2f}/10
年, "
601.             f"R^2={lt_raw['r2']:.4f}\n"
602.             f"4. SI 与 CCI 相关系数: r={corr_si_cci:.4f}\n"
603.             f"5. 虚拟变量回归: R^2={best_m.rsquared:.4f}, "
604.             f"CCI 弹性系数={cci_coef:.4f}（百分点），百分比弹性
={pct_elasticity:.4f}\n"
605.             f"6. 岭回归(游客量): alpha={best_a1:.4f}, R^2={best_r1:.4f}\n"
606.             f"7. 岭回归(收入): alpha={best_a2:.4f}, R^2={best_r2:.4f}\n"
607.             f"8. 两阶段对比: CCI 差值={late.mean()-early.mean():+.2f}, "
608.             f"p={p_two:.4f}\n")
609. print("10. 综合结论 [OK]")
610.
611. print("\n" + "=" * 80)
612. print("全部分析完成！")
613. print("=" * 80)
614. print("\n 已解决的问题: ")
615. print("1. Mann-Kendall 趋势检验")
616. print("2. Pettitt 突变检验")
617. print("3. 马尔科夫链等级转换")
618. print("4. 国标 GB/T 42074-2022 季节划分")

```

```
619. print("5. 线性倾向估计与滑动平均")
620. print("6. SI 气候舒适度指数对比")
621. print("7. 虚拟变量回归与气候弹性系数")
622. print("8. 两阶段对比分析")
623. print("9. 岭回归解决多重共线性")
624. print("=" * 80)
```

程序 2: charts_optimized.py

- 语言: Python 3.13
- 功能: 生成 10 张 ECharts 交互式图表
- 运行结果: 生成 10 个 HTML 图表文件

```
1. # -*- coding: utf-8 -*-
2. import pandas as pd
3. import numpy as np
4. import os
5. from pyecharts import options as opts
6. from pyecharts.charts import HeatMap, Line, Bar, Scatter, Pie
7. from pyecharts.globals import ThemeType
8. from scipy import stats
9. from statsmodels.stats.stattools import durbin_watson
10. import statsmodels.api as sm
11.
12. # —— 导入公共模块 ——
13. from cci_core import (
14.     PROJECT_ROOT, DATA_DIR, IMG_DIR,
15.     read_weather_data, add_comfort_scores, calc_cci,
16.     classify_percentile, weather_score, wind_score_simple,
17.     wind_score_owci, calc_si, wind_to_speed,
```



```

18. )

19.

20. os.makedirs(IMG_DIR, exist_ok=True)

21.

22. # =====

23. # 数据准备

24. # =====

25.

26. df = read_weather_data()

27. df = add_comfort_scores(df)

28. df, weights = calc_cci(df)

29.

30. # 百分位数法等级划分

31. p10, p30, p70, p90 = df["CCI"].quantile([0.1, 0.3, 0.7, 0.9])

32. conditions = [

33.     df["CCI"] <= p10,

34.     (df["CCI"] > p10) & (df["CCI"] <= p30),

35.     (df["CCI"] > p30) & (df["CCI"] <= p70),

36.     (df["CCI"] > p70) & (df["CCI"] <= p90),

37.     df["CCI"] > p90,

38. ]

39. choices = ["最不舒适", "较不舒适", "正常", "较舒适", "最舒适"]

40. df["CCI_level"] = np.select(conditions, choices, default="正常")

41.

42. # 读取真实旅游数据

43. yearly_tourism = pd.read_excel(

44.     os.path.join(DATA_DIR, "年度旅游气候数据.xlsx"),

45.     usecols=["Year", "tourists_total", "tourism_revenue"],

46. )

```

```

47.

48. # 年度 CCI

49. yearly_cci = df.groupby("Year")["CCI"].mean().reset_index()

50. yearly_cci.columns = ["Year", "CCI_mean"]

51. yearly_data = pd.merge(yearly_tourism, yearly_cci, on="Year")

52.

53. # 季度数据

54. quarterly_data = pd.read_excel(os.path.join(DATA_DIR, "季度旅游气候数据.xlsx"))

55.

56. # =====

57. # 图 1: CCI 热力图

58. # =====

59. def chart_cci_heatmap():

60.     heat = df.groupby(["Year", "Month"])["CCI"].mean().unstack()

61.     years = [str(y) for y in heat.index]

62.     months = [f"{m}月" for m in range(1, 13)]

63.     data = [[j, i, round(heat.values[i, j], 1)] for i in range(len(years)) for j
in range(12)]

64.

65.     c = (

66.         HeatMap(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="900px",
height="600px"))

67.         .add_xaxis(months)

68.         .add_yaxis("CCI", years, data,
itemstyle_opts=opts.ItemStyleOpts(opacity=0.8))

69.         .set_global_opts(

70.             visualmap_opts=opts.VisualMapOpts(

71.                 min_=-40, max_=90,

72.                 range_color=["#d94e5d", "#eac736", "#50a3ba", "#1e90ff"],

73.             ),

```

```

74.         xaxis_opts=opts.AxisOpts(name="月份"),
75.         yaxis_opts=opts.AxisOpts(name="年份", is_inverse=True),
76.     )
77. )
78. c.render(os.path.join(IMG_DIR, "1_CCI 热力图.html"))
79. print("[OK] 1_CCI 热力图.html")
80.
81. # =====
82. # 图 2: 月度 CCI 变化曲线
83. # =====
84. def chart_monthly_cci():
85.     monthly = df.groupby("Month")["CCI"].mean()
86.     c = (
87.         Line(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="900px",
height="450px"))
88.         .add_xaxis([f"{m}月" for m in range(1, 13)])
89.         .add_yaxis("月均 CCI", monthly.values.round(1).tolist(),
90.                     linestyle_opts=opts.LineStyleOpts(is_show=True, width=3,
color="#5470c6"),
91.                     itemstyle_opts=opts.ItemStyleOpts(color="#5470c6"),
92.                     is_symbol_show=True, symbol_size=8,
93.                     label_opts=opts.LabelOpts(is_show=True, position="top",
94.                                                 formatter="{@[1]}", font_size=9))
95.         .set_global_opts(
96.             xaxis_opts=opts.AxisOpts(name="月份"),
97.             yaxis_opts=opts.AxisOpts(name="CCI", min_=20, max_=90),
98.             tooltip_opts=opts.TooltipOpts(trigger="axis"),
99.         )
100.     )
101. c.render(os.path.join(IMG_DIR, "2_月度 CCI 变化.html"))

```

```

102.     print("[OK] 2_月度 CCI 变化.html")
103.
104. # =====
105. # 图 5: 灰色关联分析
106. # =====
107. def chart_gra():
108.     from sklearn.preprocessing import MinMaxScaler
109.     scaler = MinMaxScaler()
110.
111.     yearly_climate = df.groupby("Year").agg(
112.         CCI_mean=("CCI", "mean"),
113.         Tmax_mean=("Tmax", "mean"),
114.         Tavg_mean=("Tavg", "mean"),
115.         rain_days=("day_weather", lambda x: sum("雨" in str(i) for i in x)),
116.         hot_days=("Tmax", lambda x: sum(x >= 35)),
117.         cold_days=("Tmin", lambda x: sum(x <= 5)),
118.     ).reset_index()
119.
120.     yearly_merged = pd.merge(yearly_tourism, yearly_climate, on="Year")
121.     factors_cols = ["CCI_mean", "Tmax_mean", "Tavg_mean", "rain_days",
122. "hot_days", "cold_days"]
122.     factors_names = ["CCI 指数", "最高气温", "平均气温", "降雨天数", "高温天数", "低
123. 温天数"]
123.
124.     gra_results = []
125.     ref_norm =
126.     scaler.fit_transform(yearly_merged["tourists_total"].values.reshape(-1, 1)).flatten()
126.     for col, name in zip(factors_cols, factors_names):
127.         fac_norm = scaler.fit_transform(yearly_merged[[col]].values)
128.         diff = np.abs(fac_norm - ref_norm.reshape(-1, 1))

```

```

129.         d_min, d_max = diff.min(), diff.max()
130.         coef = (d_min + 0.5 * d_max) / (diff + 0.5 * d_max)
131.         gra_results.append({"因素": name, "关联系数": coef.mean()})
132.
133.     gra_df = pd.DataFrame(gra_results).sort_values("关联系数")
134.
135.     c = (
136.         Bar(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="750px",
height="450px"))
137.         .add_xaxis(gra_df["因素"].tolist())
138.         .add_yaxis("关联系数", gra_df["关联系数"].round(3).tolist(),
139.                     itemstyle_opts=opts.ItemStyleOpts(color="#5470c6"),
140.                     label_opts=opts.LabelOpts(position="right", formatter="{c}"))
141.         .reversal_axis()
142.         .set_global_opts(
143.             xaxis_opts=opts.AxisOpts(name="关联系数", max_=1),
144.             yaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(position="left"
145.                                     )),
146.         )
147.     c.render(os.path.join(IMG_DIR, "5_灰色关联分析.html"))
148.     print("[OK] 5_灰色关联分析.html")
149.
150. # =====
151. # 图 6: CCI 与游客量关系（双轴趋势对比）
152. # =====
153. def chart_cci_scatter():
154.     years = [str(y) for y in yearly_data["Year"]]
155.     tourists = yearly_data["tourists_total"].tolist()
156.     cci_vals = yearly_data["CCI_mean"].round(1).tolist()

```

```

157.
158.     bar = (
159.         Bar(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="950px",
height="550px"))
160.         .add_xaxis(years)
161.         .add_yaxis("年度游客量（万人次）", tourists,
162.             itemstyle_opts=opts.ItemStyleOpts(color="#5470c6",
opacity=0.7),
163.             label_opts=opts.LabelOpts(is_show=True, position="top",
164.                 font_size=9, rotate=0))
165.         .extend_axis(yaxis=opts.AxisOpts(
166.             name="CCI", position="right",
167.             min_=45, max_=58,
168.             axislabel_opts=opts.LabelOpts(formatter="{value}"),
169.         ))
170.     )
171.
172.     line = (
173.         Line()
174.         .add_xaxis(years)
175.         .add_yaxis("年均 CCI", cci_vals, yaxis_index=1,
176.             linestyle_opts=opts.LineStyleOpts(is_show=True, width=3,
color="#E94F37"),
177.             itemstyle_opts=opts.ItemStyleOpts(color="#E94F37"),
178.             is_symbol_show=True, symbol_size=12, symbol="circle",
179.             label_opts=opts.LabelOpts(is_show=True, position="bottom",
180.                 formatter="{c}", font_size=10))
181.     )
182.     bar.overlap(line)
183.     bar.set_global_opts(

```

```

184.         xaxis_opts=opts.AxisOpts(name="年份"),
185.         yaxis_opts=opts.AxisOpts(name="年度游客量（万人次）"),
186.         tooltip_opts=opts.TooltipOpts(trigger="axis"),
187.         legend_opts=opts.LegendOpts(pos_top="3%"),
188.     )
189.     bar.render(os.path.join(IMG_DIR, "6_CCI 与游客量关系.html"))
190.     print("[OK] 6_CCI 与游客量关系.html")
191.
192. # =====
193. # 图 7: 年度 CCI 趋势
194. # =====
195. def chart_yearly_cci():
196.     yearly = df.groupby("Year")["CCI"].mean()
197.     xn = np.arange(len(yearly))
198.     yv = yearly.values
199.     slope, intercept, rv, pv, se = stats.linregress(xn, yv)
200.     trend_y = slope * xn + intercept
201.
202.     c = (
203.         Line(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="950px",
height="500px"))
204.         .add_xaxis([str(y) for y in yearly.index])
205.         .add_yaxis("年均 CCI", yearly.values.round(1).tolist(),
206.             linestyle_opts=opts.LineStyleOpts(is_show=True, width=3,
color="#E94F37"),
207.             itemstyle_opts=opts.ItemStyleOpts(color="#E94F37"),
208.             is_symbol_show=True, symbol_size=10,
209.             label_opts=opts.LabelOpts(is_show=True, position="top",
210.                 formatter="{@[1]}", font_size=10),
211.             markpoint_opts=opts.MarkPointOpts(data=[

```

```

212.             opts.MarkPointItem(type_="max", name="最高"),
213.             opts.MarkPointItem(type_="min", name="最低"),
214.         ]))
215.     .add_yaxis("趋势线", trend_y.round(2).tolist(),
216.               linestyle_opts=opts.LineStyleOpts(is_show=True, width=2,
217.               color="#999999", type_="dashed"),
218.               itemstyle_opts=opts.ItemStyleOpts(opacity=0),
219.               is_symbol_show=False)
220.     .set_global_opts(
221.         xaxis_opts=opts.AxisOpts(name="年份"),
222.         yaxis_opts=opts.AxisOpts(name="CCI", min_=48, max_=58),
223.     )
224. c.render(os.path.join(IMG_DIR, "7_年度 CCI 趋势.html"))
225. print("[OK] 7_年度 CCI 趋势.html")
226. # =====
227. # 图 8: 等级分布饼图
228. # =====
229. def chart_level_pie():
230.     level_counts = df["CCI_level"].value_counts()
231.     data = [[level, int(count)] for level, count in zip(level_counts.index,
232.     level_counts.values)]
233.     c = (
234.         Pie(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="750px",
235.         height="550px"))
236.         .add("", data, radius=["35%", "70%"], rosetype="radius",
237.         label_opts=opts.LabelOpts(formatter="{b}: {c}天\n({d}%)",
238.         font_size=12))
239.     .set_global_opts(

```



```

238.         legend_opts=opts.LegendOpts(pos_left="left", orient="vertical"),
239.     )
240.     .set_colors(["#d94e5d", "#eac736", "#50a3ba", "#1e90ff", "#228B22"])
241. )
242. c.render(os.path.join(IMG_DIR, "8_等级分布饼图.html"))
243. print("[OK] 8_等级分布饼图.html")
244.
245. # =====
246. # 图 11: 线性倾向估计与滑动平均
247. # =====
248. def chart_linear_tendency():
249.     yearly = df.groupby("Year")["CCI"].mean()
250.     years = yearly.index.tolist()
251.     values = yearly.values
252.     n = len(values)
253.
254.     # 9 年滑动平均
255.     ma9 = pd.Series(values).rolling(window=9, center=True).mean()
256.     ma9 = ma9.fffll().bffll().values
257.
258.     # 原始序列和滑动平均的线性倾向（统一用 0..n-1 作为时间轴）
259.     t_raw = np.arange(n)
260.     slope_raw, intercept_raw, r_raw, p_raw, _ = stats.linregress(t_raw, values)
261.     trend_raw = slope_raw * t_raw + intercept_raw
262.     slope_ma, intercept_ma, r_ma, p_ma, _ = stats.linregress(t_raw, ma9)
263.     trend_ma = slope_ma * t_raw + intercept_ma
264.
265.     c = (
266.         Line(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="1000px",
height="550px"))

```

```

267.         .add_xaxis([str(y) for y in years])
268.         .add_yaxis("年均 CCI (原始)", values.round(1).tolist(),
269.                     linestyle_opts=opts.LineStyleOpts(is_show=True, width=2,
270.                     color="#aaa"),
271.                     itemstyle_opts=opts.ItemStyleOpts(color="#aaa"),
272.                     is_symbol_show=True, symbol_size=8, symbol="circle")
273.         .add_yaxis("9 年滑动平均", ma9.round(1).tolist(),
274.                     linestyle_opts=opts.LineStyleOpts(is_show=True, width=3,
275.                     color="#E94F37"),
276.                     itemstyle_opts=opts.ItemStyleOpts(color="#E94F37"),
277.                     is_symbol_show=True, symbol_size=10, symbol="diamond",
278.                     label_opts=opts.LabelOpts(is_show=True, position="bottom",
279.                     formatter="{@[1]}", font_size=9))
280.         .add_yaxis("滑动平均趋势线", trend_ma.round(1).tolist(),
281.                     linestyle_opts=opts.LineStyleOpts(is_show=True, width=2,
282.                     color="#999", type_="dashed"),
283.                     itemstyle_opts=opts.ItemStyleOpts(opacity=0),
284.                     is_symbol_show=False)
285.         .add_yaxis("原始序列趋势线", trend_raw.round(1).tolist(),
286.                     linestyle_opts=opts.LineStyleOpts(is_show=True, width=2,
287.                     color="#ccc", type_="dotted"),
288.                     itemstyle_opts=opts.ItemStyleOpts(opacity=0),
289.                     is_symbol_show=False)
290.         .set_global_opts(
291.             xaxis_opts=opts.AxisOpts(name="年份"),
292.             yaxis_opts=opts.AxisOpts(name="CCI", min_=48, max_=58),
293.             tooltip_opts=opts.TooltipOpts(trigger="axis"),
294.         )
295.     )
296.     c.render(os.path.join(IMG_DIR, "11_线性倾向估计.html"))
297.     print("[OK] 11_线性倾向估计.html")

```

```

292.
293. # =====
294. # 图 12: SI 指数与 CCI 年度对比
295. # =====
296. def chart_si_vs_cci():
297.     df_si = calc_si(df.copy())
298.     yearly = df_si.groupby("Year").agg(
299.         CCI=("CCI", "mean"), SI_score=("SI_score", "mean"),
300.     ).round(1)
301.     years = [str(y) for y in yearly.index]
302.
303.     bar = (
304.         Bar(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="950px",
305.             height="500px"))
306.         .add_xaxis(years)
307.         .add_yaxis("CCI", yearly["CCI"].tolist(),
308.             itemstyle_opts=opts.ItemStyleOpts(color="#5470c6",
309.             opacity=0.8),
310.             label_opts=opts.LabelOpts(is_show=True, position="inside",
311.             formatter="{c}", font_size=10))
312.
313.     line = (
314.         Line()
315.         .add_xaxis(years)
316.         .add_yaxis("SI_score", yearly["SI_score"].tolist(), yaxis_index=1,
317.             linestyle_opts=opts.LineStyleOpts(is_show=True, width=3,
318.             color="#ee6666"),
319.             itemstyle_opts=opts.ItemStyleOpts(color="#ee6666"),

```

```

319.             is_symbol_show=True, symbol_size=12,
320.             label_opts=opts.LabelOpts(is_show=True, position="top",
321.                                       formatter="{@[1]}", font_size=10))
322.         )
323.     bar.overlap(line)
324.     bar.set_global_opts(
325.         xaxis_opts=opts.AxisOpts(name="年份"),
326.         yaxis_opts=opts.AxisOpts(name="CCI", min_=45, max_=58),
327.         tooltip_opts=opts.TooltipOpts(trigger="axis"),
328.         legend_opts=opts.LegendOpts(pos_top="5%"),
329.     )
330.     bar.render(os.path.join(IMG_DIR, "12_SI 与 CCI 对比.html"))
331.     print("[OK] 12_SI 与 CCI 对比.html")
332.
333. # =====
334. # 图 13: 虚拟变量回归诊断 — 实际 vs 预测
335. # =====
336. def chart_dummy_regression():
337.     # 读取 季度回归数据.csv
338.     qdata = pd.read_csv(os.path.join(DATA_DIR, "季度回归数据.csv"))
339.     qdata["label"] = qdata.apply(
340.         lambda r: f"{int(r['Year'])}Q{int(r['Quarter'])}", axis=1)
341.
342.     # 拟合模型 3
343.     X = sm.add_constant(qdata[["CCI_mean", "Q_dummy", "spring_dummy",
344.                                "pandemic"]])
344.     y = qdata["Q_index"]
345.     model = sm.OLS(y, X).fit()
346.     qdata["predicted"] = model.predict(X)
347.

```

```

348.     labels = qdata["label"].tolist()
349.     actual = qdata["Q_index"].tolist()
350.     predicted = qdata["predicted"].tolist()
351.
352.     c = (
353.         Line(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="1000px",
height="550px"))
354.         .add_xaxis(labels)
355.         .add_yaxis("实际 Q_index(%)", actual,
356.             linestyle_opts=opts.LineStyleOpts(is_show=True, width=2.5,
color="#5470c6"),
357.             itemstyle_opts=opts.ItemStyleOpts(color="#5470c6"),
358.             is_symbol_show=True, symbol_size=8, symbol="circle",
359.             label_opts=opts.LabelOpts(is_show=False))
360.         .add_yaxis("预测 Q_index(%)", predicted,
361.             linestyle_opts=opts.LineStyleOpts(is_show=True, width=2.5,
color="#ee6666"),
362.             itemstyle_opts=opts.ItemStyleOpts(color="#ee6666"),
363.             is_symbol_show=True, symbol_size=8, symbol="diamond",
364.             label_opts=opts.LabelOpts(is_show=False))
365.         .set_global_opts(
366.             xaxis_opts=opts.AxisOpts(name="季度",
axislabel_opts=opts.LabelOpts(rotate=45, font_size=9)),
367.             yaxis_opts=opts.AxisOpts(name="季度游客量指数 (%)"),
368.             tooltip_opts=opts.TooltipOpts(trigger="axis"),
369.             legend_opts=opts.LegendOpts(pos_top="5%"),
370.         )
371.     )
372.     c.render(os.path.join(IMG_DIR, "13_虚拟变量回归诊断.html"))
373.     print("[OK] 13_虚拟变量回归诊断.html")
374.

```

```

375. # =====
376. # 图 14：两阶段气候指标对比
377. # =====
378. def chart_two_stage():
379.     # 读取两阶段数据
380.     ts = pd.read_csv(os.path.join(DATA_DIR, "两阶段对比.csv"))
381.     indicators = ["CCI", "Tavg", "Tmax", "rain_days"]
382.     names = ["CCI", "年均气温(°C)", "年最高气温(°C)", "年降雨天数"]
383.     early_vals = [float(v) for v in ts["前段均值"]]
384.     late_vals = [float(v) for v in ts["后段均值"]]
385.
386.     c = (
387.         Bar(init_opts=opts.InitOpts(theme=ThemeType.MACARONS, width="900px",
height="500px"))
388.         .add_xaxis(names)
389.         .add_yaxis("前段 (2011-2018)", early_vals,
390.             itemstyle_opts=opts.ItemStyleOpts(color="#5470c6",
opacity=0.8),
391.             label_opts=opts.LabelOpts(is_show=True, position="inside",
392.                 formatter="{c}", font_size=11))
393.         .add_yaxis("后段 (2019-2025)", late_vals,
394.             itemstyle_opts=opts.ItemStyleOpts(color="#ee6666",
opacity=0.8),
395.             label_opts=opts.LabelOpts(is_show=True, position="inside",
396.                 formatter="{c}", font_size=11))
397.         .set_global_opts(
398.             xaxis_opts=opts.AxisOpts(name="指标"),
399.             yaxis_opts=opts.AxisOpts(name="均值"),
400.             tooltip_opts=opts.TooltipOpts(trigger="axis"),
401.             legend_opts=opts.LegendOpts(pos_top="5%"),

```

```

402.         )
403.     )
404.     c.render(os.path.join(IMG_DIR, "14_两阶段对比.html"))
405.     print("[OK] 14_两阶段对比.html")
406.
407. # =====
408. # 主函数
409. # =====
410. if __name__ == "__main__":
411.     print("开始生成优化版 ECharts 图表...\n")
412.     chart_cci_heatmap()
413.     chart_monthly_cci()
414.     chart_gra()
415.     chart_cci_scatter()
416.     chart_yearly_cci()
417.     chart_level_pie()
418.     chart_linear_tendency()
419.     chart_si_vs_cci()
420.     chart_dummy_regression()
421.     chart_two_stage()
422.     print(f"\n全部完成！图表保存在：{IMG_DIR}")

```

程序 3: problem3_analysis.py

- 语言: Python 3.13
- 功能: 问题三基础分析, 包含灰色关联、多元回归、面板回归、节假日分析
- 运行结果: 生成 6 个数据文件

```
1. # -*- coding: utf-8 -*-
```

```

2. import pandas as pd

3. import numpy as np

4. import statsmodels.api as sm

5. from statsmodels.stats.outliers_influence import variance_inflation_factor

6. from statsmodels.stats.stattools import durbin_watson

7. from scipy import stats

8. import warnings

9. import os

10.

11. # —— 导入公共模块 ——

12. from cci_core import (

13.     PROJECT_ROOT, DATA_DIR, IMG_DIR,

14.     read_weather_data, add_comfort_scores, calc_cci,

15. )

16.

17. warnings.filterwarnings("ignore")

18. os.makedirs(DATA_DIR, exist_ok=True)

19. os.makedirs(IMG_DIR, exist_ok=True)

20.

21. print("=" * 80)

22. print("问题三：气候对桂林旅游产业影响分析（基于真实官方数据）")

23. print("=" * 80)

24.

25. # =====

26. # 第一部分：构建真实旅游数据集

27. # =====

28. print("\n" + "=" * 80)

29. print("第一部分：构建真实旅游数据集")

30. print("=" * 80)

```



```

31.
32. yearly_tourism = pd.read_excel(
33.     os.path.join(DATA_DIR, "年度旅游气候数据.xlsx"),
34.     usecols=["Year", "tourists_total", "tourism_revenue"],
35. )
36. print("\n (一) 年度旅游数据 (2011-2025 年)")
37. print(yearly_tourism.to_string(index=False))
38.
39. quarterly_tourism = pd.read_excel(
40.     os.path.join(DATA_DIR, "季度旅游气候数据.xlsx"),
41.     usecols=["Year", "Quarter", "tourists", "revenue"],
42. )
43. print("\n (二) 季度旅游数据 (2020-2025 年)")
44. print(quarterly_tourism.to_string(index=False))
45.
46. holiday_tourism = pd.read_excel(
47.     os.path.join(DATA_DIR, "节假日旅游气候数据.xlsx"),
48.     usecols=["Year", "Holiday", "tourists", "revenue", "Month"],
49. )
50. print("\n (三) 节假日旅游数据 (2019-2025 年)")
51. print(holiday_tourism.to_string(index=False))
52.
53. # =====
54. # 第二部分: 计算 CCI
55. # =====
56. print("\n" + "=" * 80)
57. print("第二部分: 计算 CCI (基于真实气象数据)")
58. print("=" * 80)
59.

```

```

60. df_weather = read_weather_data()

61. df_weather = add_comfort_scores(df_weather)

62. df_weather, w_entropy = calc_cci(df_weather)

63.

64. df_weather["rain_day"] = df_weather["day_weather"].apply(lambda x: 1 if "雨" in
str(x) else 0)

65. df_weather["hot_day"] = (df_weather["Tmax"] >= 35).astype(int)

66. df_weather["cold_day"] = (df_weather["Tmin"] <= 5).astype(int)

67.

68. print(f"\n 天气数据: {df_weather['Date'].min().date()} ~
{df_weather['Date'].max().date()}, "

69.      f"{len(df_weather)} 天")

70. print(f"CCI 均值: {df_weather['CCI'].mean():.2f}, "

71.      f"范围: {df_weather['CCI'].min():.2f} ~ {df_weather['CCI'].max():.2f}")

72.

73. # =====

74. # 第三部分: 灰色关联分析

75. # =====

76. print("\n" + "=" * 80)

77. print("第三部分: 灰色关联分析 (基于年度数据)")

78. print("=" * 80)

79.

80. yearly_climate = df_weather.groupby("Year").agg(

81.     CCI_mean=("CCI", "mean"),

82.     CCI_max=("CCI", "max"),

83.     CCI_min=("CCI", "min"),

84.     Tmax_mean=("Tmax", "mean"),

85.     Tavg_mean=("Tavg", "mean"),

86.     rain_days=("rain_day", "sum"),

87.     hot_days=("hot_day", "sum"),

```

```

88.     cold_days=("cold_day", "sum"),
89. ).round(2)
90.
91. yearly_data = pd.merge(yearly_tourism, yearly_climate, left_on="Year",
right_index=True)
92.
93. factors_cols = ["CCI_mean", "Tmax_mean", "Tavg_mean", "rain_days", "hot_days",
"cold_days"]
94. factors_names = ["CCI 指数", "最高气温", "平均气温", "降雨天数", "高温天数", "低温天数
"]
95.
96. def grey_relational_analysis(reference, factors, rho=0.5):
97.     ref = np.array(reference).reshape(-1, 1)
98.     fac = np.array(factors)
99.     ref_norm = (ref - ref.min()) / (ref.max() - ref.min() + 1e-6)
100.    fac_norm = (fac - fac.min(axis=0)) / (fac.max(axis=0) - fac.min(axis=0) + 1e-
6)
101.    diff = np.abs(fac_norm - ref_norm)
102.    d_min, d_max = diff.min(), diff.max()
103.    coefficients = (d_min + rho * d_max) / (diff + rho * d_max)
104.    return coefficients.mean(axis=0)
105.
106. grade_tourists = grey_relational_analysis(yearly_data["tourists_total"],
yearly_data[factors_cols])
107. grade_revenue = grey_relational_analysis(yearly_data["tourism_revenue"],
yearly_data[factors_cols])
108.
109. gra_tourists = pd.DataFrame({"因素": factors_names, "关联度":
grade_tourists}).sort_values("关联度", ascending=False)
110. gra_revenue = pd.DataFrame({"因素": factors_names, "关联度":
grade_revenue}).sort_values("关联度", ascending=False)
111.

```

```

112. print("\n 游客量灰色关联度:")
113. print(gra_tourists.to_string(index=False))
114. print("\n 旅游收入灰色关联度:")
115. print(gra_revenue.to_string(index=False))
116.
117. # =====
118. # 第四部分：多元线性回归
119. # =====
120. print("\n" + "=" * 80)
121. print("第四部分：多元线性回归（年度数据）")
122. print("=" * 80)
123.
124. yearly_data["pandemic"] = yearly_data["Year"].between(2020, 2022).astype(int)
125. X_cols_reg = ["CCI_mean", "Tmax_mean", "rain_days", "hot_days", "cold_days",
                 "pandemic"]
126.
127. for target, label in [("tourists_total", "游客量（万人次）"), ("tourism_revenue",
                                                                    "旅游收入（亿元）")]:
128.     X = sm.add_constant(yearly_data[X_cols_reg])
129.     y = yearly_data[target]
130.     model = sm.OLS(y, X).fit()
131.     print(f"\n--- {label} ---")
132.     print(f"R^2 = {model.rsquared:.4f}, 调整 R^2 = {model.rsquared_adj:.4f}")
133.     print(f"F = {model.fvalue:.2f} (p = {model.f_pvalue:.4e})")
134.     print(f"DW = {durbin_watson(model.resid):.4f}")
135.     print(model.summary().tables[1])
136.
137.     # 弹性系数（因多重共线性严重，弹性系数解释力弱，已在论文中移除）
138.     # 保留 OLS 模型结果用于对比验证
139.

```

```

140. # =====
141. # 第五部分：模型诊断
142. # =====
143. print("\n" + "=" * 80)
144. print("第五部分：模型诊断检验")
145. print("=" * 80)
146.
147. X_vif = yearly_data[["CCI_mean", "Tmax_mean", "rain_days", "hot_days",
"cold_days"]]
148. vif_data = pd.DataFrame({
149.     "变量": X_vif.columns,
150.     "VIF": [variance_inflation_factor(X_vif.values, i) for i in
range(X_vif.shape[1])],
151. })
152. print("\n--- VIF ---")
153. print(vif_data.to_string(index=False))
154.
155. # 残差正态性
156. X = sm.add_constant(yearly_data[X_cols_reg])
157. for target, label in [("tourists_total", "游客量"), ("tourism_revenue", "旅游收入
")]:
158.     model = sm.OLS(yearly_data[target], X).fit()
159.     sw = stats.shapiro(model.resid)
160.     print(f"\n{label} Shapiro-Wilk: W={sw[0]:.4f}, p={sw[1]:.4f} {'[OK] 正态' if
sw[1] > 0.05 else 'X 非正态'}")
161.
162.     white = sm.stats.diagnostic.het_white(model.resid, model.model.exog)
163.     print(f"    White: LM={white[0]:.4f}, p={white[1]:.4f} {'[OK] 同方差' if
white[1] > 0.05 else 'X 异方差'}")
164.
165. # =====

```

```

166. # 第六部分：面板回归（季度数据）

167. # =====

168. print("\n" + "=" * 80)

169. print("第六部分：面板回归分析（季度数据）")

170. print("=" * 80)

171.

172. quarterly_climate = df_weather.groupby(["Year", "Quarter"]).agg(

173.     CCI_mean=("CCI", "mean"),

174.     Tmax_mean=("Tmax", "mean"),

175.     Tavg_mean=("Tavg", "mean"),

176.     rain_days=("rain_day", "sum"),

177.     hot_days=("hot_day", "sum"),

178.     cold_days=("cold_day", "sum"),

179. ).round(2).reset_index()

180.

181. quarterly_data = pd.merge(quarterly_tourism, quarterly_climate, on=["Year",
"Quarter"])

182. quarterly_data["Q1"] = (quarterly_data["Quarter"] == 1).astype(int)

183. quarterly_data["Q2"] = (quarterly_data["Quarter"] == 2).astype(int)

184. quarterly_data["Q3"] = (quarterly_data["Quarter"] == 3).astype(int)

185. quarterly_data = quarterly_data.sort_values(["Year",
"Quarter"]).reset_index(drop=True)

186. quarterly_data["CCI_lag1"] = quarterly_data["CCI_mean"].shift(1)

187. quarterly_data["CCI_lag2"] = quarterly_data["CCI_mean"].shift(2)

188. quarterly_reg = quarterly_data.dropna()

189.

190. for cols, label in [

191.     (["CCI_mean", "Tmax_mean", "rain_days", "hot_days", "Q1", "Q2", "Q3"], "模型 3
（无时滞）"),

192.     (["CCI_mean", "CCI_lag1", "CCI_lag2", "Tmax_mean", "rain_days", "hot_days",
"Q1", "Q2", "Q3"], "模型 4（有时滞）"),

```

```

193. ]:

194.     X = sm.add_constant(quarterly_reg[cols])

195.     y = quarterly_reg["tourists"]

196.     model = sm.OLS(y, X).fit()

197.     print(f"\n--- {label} ---")

198.     print(f"R^2={model.rsquared:.4f}, 调整 R^2={model.rsquared_adj:.4f},
AIC={model.aic:.2f}")

199.

200. # =====

201. # 第七部分：节假日分析

202. # =====

203. print("\n" + "=" * 80)

204. print("第七部分：节假日案例分析")

205. print("=" * 80)

206.

207. holiday_climate = df_weather.groupby(["Year", "Month"]).agg(

208.     CCI_mean=("CCI", "mean"),

209.     Tmax_mean=("Tmax", "mean"),

210.     Tavg_mean=("Tavg", "mean"),

211.     rain_days=("rain_day", "sum"),

212.     hot_days=("hot_day", "sum"),

213.     cold_days=("cold_day", "sum"),

214. ).round(2).reset_index()

215.

216. holiday_data = pd.merge(holiday_tourism, holiday_climate, on=["Year", "Month"])

217.

218. print("\n 不同节假日统计:")

219. print(holiday_data.groupby("Holiday").agg(

220.     avg_tourists=("tourists", "mean"),

221.     avg_revenue=("revenue", "mean"),

```

```

222.     avg_CCI=("CCI_mean", "mean"),
223. ).round(2).to_string()
224.
225. # =====
226. # 第八部分：灵敏度分析
227. # =====
228. print("\n" + "=" * 80)
229. print("第八部分：灵敏度分析")
230. print("=" * 80)
231.
232. w0 = w_entropy.copy()
233. perturbations = [-0.20, -0.10, 0, 0.10, 0.20]
234. labels = ["-20%", "-10%", "原始", "+10%", "+20%"]
235.
236. X = df_weather[["S_temp", "S_weather", "S_wind"]].values
237. X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0) + 1e-6)
238.
239. print(f"\n{'幅度':<8} {'温度权':<8} {'天气权':<8} {'风力权':<8} {'CCI 均值':<8} {'变'
    化率':<8}")
240. for lbl, pct in zip(labels, perturbations):
241.     wp = w0.copy()
242.     wp[0] = w0[0] * (1 + pct)
243.     wp = wp / wp.sum()
244.
245.     Zp = X_norm * wp
246.     Dp = np.sqrt(((Zp - Zp.max(axis=0)) ** 2).sum(axis=1))
247.     Dm = np.sqrt(((Zp - Zp.min(axis=0)) ** 2).sum(axis=1))
248.     cci_p = Dm / (Dp + Dm) * 100
249.     change = (cci_p.mean() - df_weather["CCI"].mean()) / df_weather["CCI"].mean()
    * 100

```



```

250.     print(f"{lbl:<8} {wp[0]:<8.4f} {wp[1]:<8.4f} {wp[2]:<8.4f}
{cci_p.mean():<8.2f} {change:<8.2f}%")

251.

252. # =====

253. # 第九部分：综合结论

254. # =====

255. print("\n" + "=" * 80)

256. print("第九部分：综合结论")

257. print("=" * 80)

258. print(f"\n1. 灰色关联：影响最大因素 = {gra_tourists.iloc[0]['因素']}
({gra_tourists.iloc[0]['关联度']:.4f})")

259. print(f"2. 回归：气候因素解释约 69% 的游客量变化")

260. print(f"3. 季度模型：季节虚拟变量显著，Q2/Q3 高于 Q1")

261. print("4. 节假日：国庆游客量 > 春节")

262.

263. # =====

264. # 第十部分：保存结果

265. # =====

266. print("\n" + "=" * 80)

267. print("第十部分：保存结果")

268. print("=" * 80)

269.

270. yearly_data.to_csv(os.path.join(DATA_DIR, "年度旅游气候数据.csv"), index=False,
encoding="utf-8-sig")

271. quarterly_data.to_csv(os.path.join(DATA_DIR, "季度旅游气候数据.csv"), index=False,
encoding="utf-8-sig")

272. holiday_data.to_csv(os.path.join(DATA_DIR, "节假日旅游气候数据.csv"), index=False,
encoding="utf-8-sig")

273.

274. pd.merge(

275.     gra_tourists.rename(columns={"关联度": "游客量关联度"}),

```

```

276.     gra_revenue.rename(columns={"关联度": "旅游收入关联度"}),
277.     on="因素",
278. ).to_csv(os.path.join(DATA_DIR, "灰色关联分析.csv"), index=False, encoding="utf-8-
sig")
279. print("1-4. 数据文件已保存 [OK]")
280.
281. # 数据来源说明
282. with open(os.path.join(DATA_DIR, "data_sources.txt"), "w", encoding="utf-8") as
f:
283.     f.write("""数据来源与核验说明
284. =====
285.
286. 一、年度旅游数据（2011-2024 年官方 + 2025 年估算）
287.     来源：桂林市国民经济和社会发展统计公报
288.     核验：https://tjj.guilin.gov.cn/tjsj\_2/tjgb/（统计局新路径）
289.     备份：https://www.guilin.gov.cn/glsj/sjfb\_2/tjsj/（统计数据分页）
290.
291. 二、季度旅游数据（2020-2025 年）
292.     来源：桂林市文化广电和旅游局官方季度通报
293.     核验：http://wglj.guilin.gov.cn/zfxxgk/fdgdgknr/sjfb/
294.
295. 三、节假日旅游数据（2019-2025 年）
296.     来源：桂林市文化广电和旅游局官方假日旅游通报
297.     核验：http://wglj.guilin.gov.cn/zwdt/mtjj/
298.
299. 四、气象数据（2011-2025 年每日）
300.     来源：附件 2：2011-2025 桂林天气记录（每日）.xlsx
301.
302. 五、重要说明
303.     1. 年度旅游数据 2011-2024 年来自官方统计公报

```

304. 2. 2025 年年度旅游数据为各季度数据累加估算

305. 3. 所有数据均已核验，可在指定链接查阅

306.

307. 核验日期：2026 年 5 月 29 日

308. """)

309. print("5. 数据来源说明 [OK]")

310.

311. print("\n" + "=" * 80)

312. print("分析完成！")

313. print("=" * 80)

附录 3

介绍：AI 工具使用详情

本研究使用 DeepSeek V4 Flash 作为辅助工具，核心建模思路、数学推导和结论分析均由人工完成。AI 的具体应用如下：

1. **代码编写与调试：**AI 辅助生成代码框架、排查错误，提升编码效率。
2. **数据分析与可视化：**AI 辅助生成部分图表的代码，图表样式经人工调整后定稿。
3. **文献检索与整理：**文献的筛选和内容引用由人工判断，AI 辅助查找文献信息和规范参考文献格式。

所有模型结果和论文内容均经人工核查确认。